

Прикарпатський національний університет імені Василя Стефаника
Кафедра інформаційних технологій

УДК 004.05

КВАЛІФІКАЦІЙНА РОБОТА
на здобуття першого (бакалаврського) рівня вищої освіти

Тема: Розробка програмного забезпечення системи підготовки наукових видань

Освітня програма Інженерія програмного забезпечення
назва освітньої програми
Спеціальність 121 Інженерія програмного забезпечення
код і назва спеціальності

ПОЯСНЮВАЛЬНА ЗАПИСКА
КР.ІІЗ-4220.ІІЗ
(позначення)

Рецензент

доц. _____ .2025 Любо́в КОСТИШИН
(посада) (підпис) (дата) (розшифровка підпису)

Здобувач вищої освіти

ІІЗ-41 _____ .2025 Мар'ян СТАРОВОЙТОВ
(шифр групи)(підпис) (дата) (розшифровка підпису)

Нормоконтролер

доц. _____ .2025 Любо́в КОСТИШИН
(посада) (підпис) (дата) (розшифровка підпису)

Керівник кваліфікаційної роботи

проф. _____ .2025 Бори́с НЕЗАМАЙ
(посада) (підпис) (дата) (розшифровка підпису)

Допускається до захисту

Завідувач кафедри

_____ .06.2025 Бори́с НЕЗАМАЙ
(підпис) (дата) (розшифровка підпису)

2025
(рік)

Прикарпатський національний університет імені Василя Стефаника
Факультет математики та інформатики Кафедра інформаційних технологій
Освітня програма Інженерія програмного забезпечення
Спеціальність 121 Інженерія програмного забезпечення

Завідувач кафедри

„23” жовтня 2024 р.

ЗАВДАННЯ НА ВИКОНАННЯ КВАЛІФІКАЦІЙНОЇ РОБОТИ

Старовойтову Мар'яну Андрійовичу

(прізвище, ім'я, по батькові здобувача освіти)

1. Тема роботи: “Розробка програмного забезпечення системи підготовки наукових видань”

затверджена розпорядженням по факультету математики та інформатики від „23 жовтня” 2024 р. №4

2. Термін здачі здобувачем вищої освіти закінченої роботи: 29 травня 2025 р.

3. Вихідні дані до кваліфікаційної роботи: Стандарт кафедри Інформаційних Технологій ПНУ “Вимоги до змісту та оформлення”

4. Зміст пояснювальної записки (перелік питань, що їх належить опрацювати):

1 Аналіз предметної області та постановка задачі

2 Проектування задачі

3 Розробка системи підготовки наукових видань

4 Бізнес план та витрати на систему

5. Перелік графічного матеріалу (з точним забезпеченням обов'язкових креслень): “Мета роботи”, “Актуальність теми”, “Постановка задачі”, “Аналіз предметної області”, “Огляд існуючих платформ”, “Формування вимог до системи”, “Проектування бази даних”, “Визначення ролей користувачів”, “Розробка backend частини”, “Розробка frontend інтерфейсу”, “Реалізація API та механізмів безпеки”, “Інтеграція з зовнішніми сервісами”, “Контейнеризація системи”, “Бізнес-план”, “Висновки”

6. Дата видачі завдання:

23.10.2024 p.

Керівник

(підпис)

Борис НЕЗАМАЙ

(розшифровка підпису)

Завдання прийняв до виконання

(підпис)

Мар'ян СТАРОВОЙТОВ

(розшифровка підпису)

КАЛЕНДАРНИЙ ПЛАН
написання кваліфікаційної роботи здобувачами вищої освіти
РВО бакалавр спеціальності 121 Інженерія програмного забезпечення

№	Назва етапів кваліфікаційної роботи	Термін*	Примітка
1.	Затвердження теми і керівника кваліфікаційної роботи (далі “роботи”) на засіданні випускової кафедри і Вченій раді факультету	25.10.2024	Виконано
2.	Складання та затвердження індивідуального завдання та формування плану роботи з науковим керівником	25.10.2024	Виконано
3.	Обґрунтування актуальності, формулювання мети, завдання, предмету та об’єкту дослідження. Підготовка вступу.	10.11.2024	Виконано
4.	Узгодження орієнтовних тем, виду та термінів подачі до друку результатів дослідження.	20.11.2024	Виконано
5.	Опрацювання джерел з теми роботи. Підготовка I розділу роботи.	15.12.2024	Виконано
6.	Виправлення зауважень. Підготовка II розділу роботи.	15.01.2025	Виконано
7.	Виправлення зауважень. Підготовка III розділу роботи.	15.02.2025	Виконано
8.	Виправлення зауважень. Підготовка IV розділу роботи.	15.03.2025	Виконано
9.	Виправлення зауважень. Підготовка висновків.	15.04.2025	Виконано
10.	Оформлення роботи згідно вимог.	01.05.2025	Виконано
11.	Виправлення зауважень. Надсилання науковим керівником роботи нормоконтролеру.	15.05.2025	Виконано
12.	Перевірка роботи нормоконтролером.	22.05.2025	Виконано
13.	Виправлення зауважень нормоконтролера. Надсилання роботи випусковою кафедрою відповідальній особі від факультету для перевірки на академічний плагіат.	29.05.2025	Виконано
14.	Попередній захист роботи на випусковій кафедрі та допуск її до захисту в екзаменаційній комісії.	за 15 днів до захисту	Виконано
15.	Подача роботи із усіма необхідними матеріалами на випускову кафедру	за 10 днів до захисту	Виконано
16.	Рецензування.	за 5 днів до захисту	Виконано
17.	Захист роботи	26.06.2025	Виконано

Здобувач

(підпис)

Мар'ян СТАРОВОЙТОВ

(Власне ім'я та ПРІЗВИЩЕ)

Керівник роботи

(підпис)

Борис НЕЗАМАЙ

(Власне ім'я та ПРІЗВИЩЕ)

РЕФЕРАТ

Пояснювальна записка: 89 сторінок, 11 рисунків, 3 таблиці, 23 джерела, 3 додатки.

КЛЮЧОВІ СЛОВА: наукове видавництво, система публікацій, NestJS, PostgreSQL, React, API, рецензування.

Об'єкт дослідження – процес організації та підтримки електронного наукового публікування з використанням інформаційних систем.

Мета роботи – спроектувати та реалізувати програмне забезпечення, що забезпечує повний цикл наукового публікування, включаючи подання статей, рецензування, редагування, затвердження та публікацію, із дотриманням міжнародних стандартів.

Стислий опис тексту пояснювальної записки – у першому розділі проаналізовано предметну область, визначено функціональні та не функціональні вимоги. У другому проведено проектування архітектури системи та бази даних, описано ролі користувачів та бізнес-процеси. У третьому розділі реалізовано вебзастосунок із клієнтською та серверною частинами, REST API та основні функції системи, у четвертому розділі сформовано бізнес-план та наведено висновки щодо ефективності системи та витрат на неї.

ABSTRACT

Explanatory note: 89 pages, 11 figures, 3 tables, 23 references, 3 appendices.

KEYWORDS: scientific publishing, publication system, NestJS, PostgreSQL, React, API, peer review.

Object of the study – the process of organizing and supporting electronic scientific publishing through the use of information systems.

Purpose of the work – to design and develop software that supports the full cycle of scientific publishing, including article submission, peer review, editing, approval, and publication, in accordance with international standards.

Summary of the explanatory note – the first chapter analyzes the subject area and defines functional and non-functional requirements. The second chapter presents the system architecture and database design, describes user roles and business processes. The third chapter provides the implementation of a web application with client and server parts, a REST API, and the core system functionality. The fourth chapter presents a business plan and conclusions regarding the system's efficiency and associated costs.

ВСТУП.....	8
1 АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ПОСТАНОВКА ЗАДАЧІ	10
1.1 Проблематика наукового публікування	10
1.2 Аналіз існуючих рішень.....	11
1.3 Вимоги до сучасної системи наукового видавництва.....	15
1.4 Постановка задачі, функціональні та нефункціональні вимоги	18
2 ПРОЕКТУВАННЯ СИСТЕМИ	21
2.1 Вибір технологій.....	21
2.2 Архітектура системи	25
2.2.1 Загальна архітектура системи.....	25
2.2.2 Архітектура клієнтської частини	26
2.2.3 Архітектура серверної частини	28
2.2.4 Взаємодія компонентів системи.....	29
2.2.5 Безпека та масштабованість	30
2.3 Проектування схеми бази даних	31
2.4 Визначення ролей користувачів та бізнес-процесів.....	41
2.5 Опис API (основні ендпоінти, маршрути, методи)	43
2.5.1 Філософія побудови API.....	43
2.5.2 Організація маршрутів та ресурсів	43
2.5.3 Безпека, контроль доступу та валідація	45
2.5.4 Документування та інтерактивність	45
2.5.5 Версіонування API та backward compatibility	46
2.5.6 Інтеграція з зовнішніми сервісами.....	46
2.5.7 Масштабованість, тестування та розвиток.....	46
2.5.8 Практичні сценарії використання та інструментарій.....	47
3 РОЗРОБКА СИСТЕМИ ПІДГОТОВКИ НАУКОВИХ ВИДАНЬ	48
3.1 Опис структури розробки проекту.....	48
3.1.1 Архітектурні принципи та підхід до структурування коду	48
3.1.2 Логіка організації коду.....	50
3.2 Реалізація основних модулів	52
3.3 Інтеграція системи авторизації та аутентифікації	57
3.3.1 Конфігурація JwtModule та стратегія	57
3.3.2 Захист маршрутів через guards та рольова авторизація	58
3.3.3 Взаємодія AuthService з UsersService	59
3.3.4 Процес логіну, оновлення токена, перевірки доступу	59
3.4 Приклади сценаріїв та REST API.....	60
3.5 Створення інтерфейсу системи	64

3.5.1 Технологічний стек фронтенд-частини	64
3.5.2 Архітектура фронтенд-частини	64
3.5.3 Основні компоненти системи	65
3.5.4 Взаємодія з API	66
3.5.5 Особливості користувацького інтерфейсу	67
3.5.6 Забезпечення безпеки	68
3.5.7 Результати розробки інтерфейсу	68
3.6 Контейнеризація системи	69
3.6.1 Вступ до контейнеризації	69
3.6.2 Multi-stage building	70
3.6.3 Контейнеризація бекенд-частини	70
3.6.4 Контейнеризація фронтенд-частини	71
3.6.5 Оркестрація контейнерів з Docker Compose	72
4 БІЗНЕС-ПЛАН ТА ВИТРАТИ НА СИСТЕМУ	75
4.1 Аналіз ринку та конкурентів	75
4.2 Цільова аудиторія, сегментація та позиціонування	78
4.3 Оцінка витрат на розробку і підтримку	81
ВИСНОВКИ	85
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ	87
ДОДАТОК А	89
ДОДАТОК Б	90
ДОДАТОК В	92

ВСТУП

У сучасних умовах розвитку інформаційного суспільства вагомого значення набуває організація ефективних процесів наукової комунікації, зокрема у сфері публікації результатів досліджень. Науково-видавничі системи стають ключовим елементом інфраструктури, що забезпечує прозорість, зручність і швидкість оприлюднення наукових досягнень.

Дедалі частіше наукові колективи, редакції журналів та дослідники стикаються з необхідністю автоматизації процесів подання, рецензування, редагування та публікації статей. Застарілі моделі, що базуються на ручному пересиланні матеріалів електронною поштою чи використанні розрізнених інструментів, не відповідають сучасним вимогам відкритості, оперативності й якості наукового обміну.

У світі існує чимало готових платформ для організації електронних редакцій, проте більшість із них або недостатньо адаптовані під специфіку українського наукового простору, або мають закритий код, обмежену функціональність, складний інтерфейс та високі вимоги до технічного супроводу. Ці фактори істотно обмежують можливості малих та середніх наукових колективів, а також ускладнюють впровадження кращих практик відкритої науки, прозорого рецензування і цифрової ідентифікації авторів та рецензентів.

Водночас стрімкий розвиток веб-технологій, розширення можливостей хмарних сервісів, інтенсивне впровадження RESTful API та сучасних підходів до розробки бекенд-систем відкривають нові горизонти для створення гнучких, масштабованих і безпечних науково-видавничих платформ. Особливого значення набуває побудова таких систем на основі відкритих стандартів, із застосуванням модульної архітектури, що дозволяє швидко розширювати

					КР.ІІЗ-4220.ІІЗ	Арк.
						8
Зм.	Арк.	№ докум.	Підпис	Дата		-

функціонал, інтегрувати зовнішні сервіси, забезпечувати високий рівень безпеки й захисту персональних даних.

Мета дипломної роботи полягає у розробці повнофункціональної науково-видавничої системи, яка охоплює всі ключові етапи роботи електронного наукового журналу – від реєстрації користувачів та подання рукописів до організації рецензування, прийняття рішень редакторами, публікації статей та супровідної комунікації між учасниками процесу. У роботі зроблено акцент на побудову сучасної бекенд-архітектури із застосуванням фреймворку NestJS, що відповідає найкращим галузевим практикам й забезпечує просте масштабування системи, захист даних і високу продуктивність.

Завданням дослідження є не лише розробка програмного забезпечення, а й комплексний аналіз предметної області, формалізація бізнес-процесів наукового видавництва, проектування оптимальної структури даних, а також розробка бізнес-плану подальшого впровадження і розвитку системи. У межах роботи передбачено детальне обґрунтування вибору технічних рішень, опис архітектурних підходів, аналіз конкурентного середовища, визначення цільової аудиторії та моделі монетизації.

Структура роботи відповідає логіці повного циклу розробки науково-видавничої системи: від теоретичних основ і аналізу вимог до проектування, реалізації, тестування та обґрунтування бізнес-перспектив. Такий підхід дозволяє не лише продемонструвати технічну компетентність у сфері створення складних веб-систем, а й показати розуміння економічних, організаційних та маркетингових аспектів успішного впровадження програмного продукту в реальному секторі науки та освіти.

					КР.ІІЗ-4220.ІІЗ	Арк.
						9
Зм.	Арк.	№ докум.	Підпис	Дата		-

1 АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ПОСТАНОВКА ЗАДАЧІ

1.1 Проблематика наукового публікування

Система наукової комунікації є фундаментальним елементом розвитку сучасної науки. Від ефективності механізмів обміну знаннями, результатами досліджень та рецензування залежить не лише якість наукового продукту, а й швидкість впровадження нових ідей у практику. Протягом останніх десятиліть модель наукового публікування зазнала суттєвих змін, однак донині залишається низка актуальних проблем, що стримують розвиток як окремих дослідницьких спільнот, так і світової науки загалом.

Однією з ключових проблем є громіздкість і неефективність традиційних процедур взаємодії між авторами, рецензентами та редакціями наукових журналів. Досі значна частина комунікації здійснюється електронною поштою, що призводить до втрати інформації, затримок у розгляді рукописів, дублювання запитів тощо. Автори часто не мають змоги оперативно відстежувати статус поданих робіт, а редактори – контролювати всі етапи руху рукопису в межах редакційного процесу.

Ще однією суттєвою проблемою є закритість і непрозорість процедур рецензування. У багатьох наукових журналах процес рецензування залишається анонімним і часто недостатньо структурованим, що ускладнює забезпечення якості та неупередженості оцінки наукових матеріалів. У свою чергу, це породжує сумніви щодо об'єктивності рішень, а іноді навіть спричиняє випадки академічної недоброчесності.

Невирішеною залишається й проблема доступності наукової інформації. Значна частина результатів досліджень публікується у виданнях із платним доступом або у вигляді закритих PDF-файлів, що обмежує можливість використання наукових знань широким колом зацікавлених фахівців, студентів,

					КР.ІІЗ-4220.ІІЗ	Арк.
						10
Зм.	Арк.	№ докум.	Підпис	Дата		-

представників суміжних галузей. Це суперечить принципам відкритої науки, які передбачають максимально вільний обмін знаннями та результатами досліджень.

Варто також відзначити проблему ідентифікації та обліку авторських прав. Відсутність єдиних механізмів ідентифікації авторів, рецензентів та редакторів ускладнює ведення електронних архівів, формування наукометричних показників, а також створює додаткові ризики для захисту інтелектуальної власності.

Окремої уваги потребують питання безпеки та захисту персональних даних. З огляду на те, що процес наукового публікування передбачає обробку значного обсягу особистої інформації, рукописів, рецензій та службового листування, важливою є розробка і впровадження сучасних засобів захисту даних – як на технічному, так і на організаційному рівнях.

Крім того, дедалі більшого значення набувають питання автоматизації та стандартизації процесів. Відсутність єдиних стандартів опису метаданих, механізмів імпорту/експорту робіт, а також інтеграції з міжнародними реєстрами (наприклад, ORCID, DOI) [4] ускладнює обіг наукової інформації і гальмує розвиток глобального наукового простору.

З огляду на перелічені проблеми, розвиток ефективних науково-видавничих систем, що дозволяють автоматизувати, стандартизувати й зробити прозорими всі ключові етапи підготовки, рецензування й публікації наукових матеріалів, є одним з пріоритетних завдань сучасного інформаційного суспільства. Саме цим питанням і присвячено дану дипломну роботу.

1.2 Аналіз існуючих рішень

На сучасному етапі розвитку наукової комунікації цифрові платформи для організації процесу публікації наукових матеріалів стали невід’ємною частиною

					КР.ІІЗ-4220.ІІЗ	Арк.
						11
Зм.	Арк.	№ докум.	Підпис	Дата		-

функціонування академічних журналів, видавництв і дослідницьких організацій.

Ринок редакційних систем надзвичайно динамічний та насичений: упродовж останніх десятиліть з'явилося чимало як універсальних, так і вузькоспеціалізованих платформ, які автоматизують ключові етапи редакційного workflow – від первинного подання рукопису, рецензування та прийняття рішень до остаточного опублікування й розповсюдження результатів досліджень у глобальному науковому просторі.

Серед найбільш відомих та розповсюджених рішень у цій сфері вирізняються такі платформи, як Open Journal Systems (OJS), ScholarOne Manuscripts, Editorial Manager, Manuscript Central, а також низка менш масштабних, але популярних у певних галузях чи регіонах систем. Кожна з цих платформ має свої унікальні риси, функціональні можливості, переваги й, відповідно, певні обмеження, які необхідно враховувати при виборі або створенні нової редакційної системи [2, 3].

Open Journal Systems (OJS), розроблена ініціативою Public Knowledge Project (PKP), стала однією з найбільш поширених open-source платформ для наукових журналів у світі. OJS забезпечує повноцінний супровід усіх етапів роботи з науковими статтями: реєстрацію авторів, подання рукописів, призначення рецензентів, управління процесом рецензування, прийняття рішень редакторами, підготовку до публікації, верстку та архівування матеріалів.

Система активно підтримується спільнотою, має велику кількість локалізацій, гнучко налаштовується під різні моделі організації наукових журналів. Її відкритий код дозволяє розширювати функціонал через плагіни, адаптувати під специфічні потреби окремих видань, а також забезпечує інтеграцію з багатьма зовнішніми сервісами.

Проте, попри універсальність та популярність, OJS має і суттєві недоліки. Інтерфейс платформи часто сприймається користувачами як застарілий і недостатньо інтуїтивний, що може ускладнювати роботу редакцій та

					КР.ІІЗ-4220.ІІЗ	Арк.
						12
Зм.	Арк.	№ докум.	Підпис	Дата		-

відштовхувати молодих авторів і рецензентів. Налаштування OJS інколи вимагають значних технічних знань, що створює додаткові бар'єри для невеликих редакцій, які не мають власних IT-фахівців. Інтеграція з сучасними сервісами цифрових ідентифікаторів (DOI, ORCID, Crossref) [5], системами антиплагіату чи наукометричними базами потребує окремих модулів і часто є складною для впровадження. Оновлення версій OJS нерідко призводять до втрати сумісності з наявними плагінами, що ускладнює супровід і вимагає додаткових ресурсів.

Іншою масштабною системою є ScholarOne Manuscripts – продукт компанії Clarivate Analytics, який зорієнтований переважно на великі комерційні видавництва та провідні наукові журнали. ScholarOne відома своєю потужною багаторівневою системою ролей, гнучкими сценаріями workflow, розвиненими інструментами для аналітики, звітності, інтеграції з міжнародними наукометричними базами. Проте ця система є закритою та комерційною: її впровадження і супровід коштують значних фінансових ресурсів, а налаштування вимагає участі висококваліфікованих технічних спеціалістів. Для невеликих чи регіональних видань ScholarOne часто є економічно недоцільною, а її складність перевищує реальні потреби локальних редакцій.

Ще одним яскравим прикладом комерційної платформи преміум-класу є Editorial Manager (Aries Systems), яка широко використовується у видавництвах світового рівня. Система відзначається гнучкістю налаштувань, підтримкою різноманітних сценаріїв рецензування (зокрема, подвійного сліпого рецензування), інтеграцією з глобальними інфраструктурними сервісами та розвинутою системою сповіщень. Однак впровадження Editorial Manager пов'язане з високими витратами, складністю технічного супроводу й вимогами до навченого персоналу, що знову ж таки обмежує її доступність для невеликих або некомерційних журналів.

					КР.ІІЗ-4220.ІІЗ	Арк.
						13
Зм.	Арк.	№ докум.	Підпис	Дата		-

На ринку представлені й інші рішення, такі як Manuscript Manager, eJournalPress, PeerTrack, які розраховані на вузькі сегменти ринку або мають обмежений функціонал. Багато редакцій, особливо у країнах, що розвиваються, або у невеликих наукових співтовариствах, використовують власноруч розроблені системи, створені під конкретні потреби. Однак такі “самописні” рішення, зазвичай, не мають належної документації, не масштабуються, складні у підтримці, швидко морально старіють і не дозволяють інтегрувати сучасні сервіси та інструменти.

За останні роки спостерігається тенденція переходу до хмарних сервісів і SaaS-моделей (Software as a Service), коли видавництва та журнали орендують доступ до платформи, отримуючи автоматичні оновлення, резервне копіювання, технічну підтримку. Такий підхід зменшує навантаження на локальні ІТ-служби, дозволяє швидко масштабувати інфраструктуру відповідно до зростання кількості користувачів чи обсягу даних. Водночас, SaaS-рішення породжують нові виклики: питання юрисдикції й безпеки зберігання персональних даних, можливість гнучкого налаштування під специфіку конкретного журналу, залежність від політики постачальника послуг.

Особливості українського наукового середовища додають специфічних проблем: більшість журналів досі використовують застарілі локальні рішення, які роками не оновлювалися й не відповідають сучасним вимогам до відкритої науки, цифрової ідентифікації, захисту персональних даних. Додатково редакції часто стикаються з браком кваліфікованих ІТ-спеціалістів, проблемами локалізації інтерфейсу, відсутністю інтеграції з українськими та міжнародними реєстрами, а також низькою гнучкістю архітектури, що унеможливорює оперативне впровадження нових бізнес-процесів чи змін у законодавстві.

Одним із найважливіших аспектів, який часто ігнорується у багатьох платформах, є якість користувацького досвіду. Наявність складного, перевантаженого або застарілого інтерфейсу, відсутність адаптації під мобільні

					КР.ІІЗ-4220.ІІЗ	Арк.
						14
Зм.	Арк.	№ докум.	Підпис	Дата		-

пристрої, низька швидкість роботи, незручна навігація – все це негативно позначається на залученості авторів, рецензентів, редакторів, знижує ефективність роботи й гальмує науковий процес загалом.

Аналізуючи існуючі рішення, можна зробити висновок, що жодна з представлених платформ не задовольняє повною мірою вимоги сучасної наукової спільноти, особливо у світлі тенденцій до відкритої науки, цифрової трансформації, широкої інтеграції з глобальними сервісами, підвищення вимог до безпеки та захисту персональних даних [3]. Саме ці обставини формують актуальний запит на створення нових, гнучких, масштабованих та інноваційних систем, які відповідають стандартам відкритості, підтримують сучасні технології, легко інтегруються з зовнішніми сервісами, є доступними для локальних редакцій і сприяють підвищенню конкурентоспроможності українських та міжнародних наукових журналів.

1.3 Вимоги до сучасної системи наукового видавництва

У сучасних умовах розвитку науки та інформаційних технологій до систем наукового видавництва висуваються значно вищі вимоги, ніж ще кілька років тому. Це обумовлено як загальносвітовими тенденціями у сфері відкритої науки [2], так і зміною очікувань користувачів, посиленням конкуренції між журналами та зростанням ролі цифрових ідентифікаторів, автоматизованих сервісів та аналітики. Вимоги до платформи не обмежуються лише простотою публікації статей; вони охоплюють весь цикл взаємодії між авторами, редакторами, рецензентами, адміністраторами та читачами.

Першочерговою вимогою є забезпечення повного циклу наукового публікування – від реєстрації користувача, подання рукопису, проходження етапів рецензування, внесення правок, затвердження та публікації до подальшої аналітики та взаємодії з міжнародними реєстрами. Система повинна підтримувати різні ролі користувачів із чітко визначеними правами доступу та

					КР.ІІЗ-4220.ІІЗ	Арк.
						15
Зм.	Арк.	№ докум.	Підпис	Дата		-

сценаріями взаємодії: автор, рецензент, редактор, адміністратор, технічний секретар тощо. Надзвичайно важливою є можливість гнучкого налаштування ролей, делегування повноважень, а також забезпечення прозорості дій кожного учасника процесу для підвищення якості наукової комунікації [19].

Значну увагу приділяють підтримці різних моделей рецензування: відкритого, закритого, подвійного сліпого, анонімного тощо. Це дозволяє редакціям адаптувати систему під власну політику і стандарти, а також забезпечувати довіру з боку наукової спільноти. Окрім стандартного workflow, сучасна система має підтримувати гнучке налаштування етапів рецензування, можливість залучення зовнішніх експертів, формування шаблонів рецензій та автоматичне надсилання сповіщень усім учасникам процесу.

Ще однією важливою вимогою є інтеграція із зовнішніми сервісами та базами даних, такими як Crossref, ORCID, Publons, DOI, Google Scholar, Scopus, Web of Science. Система повинна забезпечувати автоматичне формування та реєстрацію цифрових ідентифікаторів для статей і авторів, експортувати метадані у форматах, сумісних із міжнародними стандартами, та підтримувати інтеграцію з антиплагіатними сервісами. Це значно підвищує рівень видимості й цитованості журналу, а також спрощує введення даних у різноманітні наукометричні системи.

З огляду на вимоги відкритої науки, система повинна підтримувати політику відкритого доступу, надавати зручні інструменти для ліцензування публікацій (зокрема Creative Commons), а також забезпечувати збереження та розповсюдження дослідницьких даних, супровідних матеріалів, відкритих рецензій. Система має інтегруватися з репозитаріями відкритих даних, підтримувати багатомовність, генерувати бібліографічні посилання у різних форматах та автоматично формувати індекси.

Надзвичайно високі вимоги висуваються до безпеки та захисту персональних даних. Оскільки платформа обробляє значний обсяг

					КР.ІІЗ-4220.ІІЗ	Арк.
						16
Зм.	Арк.	№ докум.	Підпис	Дата		-

конфіденційної інформації (особисті дані користувачів, тексти рукописів, службові документи, приватне листування), вона повинна відповідати міжнародним стандартам захисту інформації (GDPR, ISO/IEC 27001), забезпечувати багаторівневу аутентифікацію, шифрування даних, аудит дій користувачів, резервне копіювання та швидке відновлення у разі аварійних ситуацій.

Важливим аспектом є забезпечення зручного, сучасного та інтуїтивно зрозумілого інтерфейсу як для авторів, так і для редакторів, рецензентів та адміністраторів. Інтерфейс системи повинен бути адаптивним, підтримувати роботу з мобільних пристроїв, забезпечувати швидкий доступ до основних функцій, мати продуману систему сповіщень та підказок. Особливо актуальною є підтримка роботи з великими обсягами даних, швидкий пошук і фільтрація, а також можливість масового імпорту та експорту інформації.

Окремо слід визначити вимоги до масштабованості та надійності. Система повинна легко адаптуватися до зростання кількості користувачів, обсягу бази даних, кількості журналів чи публікацій, підтримувати роботу у кластерних або хмарних середовищах, мати можливість горизонтального та вертикального масштабування, балансування навантаження, автоматичного оновлення компонентів без зупинки сервісу.

Вагомим є і питання аналітики та звітності. Кожен учасник процесу має отримувати доступ до релевантних звітів: автори – щодо статусу своїх рукописів, редактори – по завантаженню журналу, рецензенти – по історії рецензій, адміністратори – щодо активності системи, кількості публікацій, тематики, географії авторів тощо. Аналітика має бути інтерактивною, з можливістю гнучкого фільтрування та експорту даних для подальшого аналізу [5].

Нарешті, до сучасної системи висуваються вимоги щодо відкритості коду, можливості розширення функціоналу, створення власних модулів чи плагінів, інтеграції з іншими системами через API, а також оперативної технічної

					КР.ІІЗ-4220.ІІЗ	Арк.
						17
Зм.	Арк.	№ докум.	Підпис	Дата		-

підтримки, детальної документації та широкої спільноти користувачів і розробників.

Таким чином, сучасна система наукового видавництва – це комплексне, багаторівневе середовище, яке повинно одночасно забезпечувати автоматизацію всіх ключових бізнес-процесів наукового журналу, відповідати міжнародним стандартам, забезпечувати безпеку, зручність, масштабованість і підтримувати інтеграцію із зовнішніми інфраструктурними сервісами. Тільки такі системи мають потенціал стати основою для успішного функціонування і розвитку наукової комунікації у XXI столітті.

1.4 Постановка задачі, функціональні та нефункціональні вимоги

Враховуючи сучасний стан розвитку систем наукового видавництва й проаналізовані проблеми та вимоги, постає завдання розробити універсальну, гнучку й масштабовану платформу, яка дозволить автоматизувати весь життєвий цикл наукової публікації. Основним завданням є створення програмного продукту, здатного охопити всі ключові аспекти редакційної діяльності наукового журналу, забезпечити високий рівень безпеки, прозорості та інтегрованості із зовнішніми сервісами.

Однією з головних функціональних вимог є реалізація багаторівневої моделі ролей користувачів, де кожен учасник процесу – автор, рецензент, редактор, адміністратор – має чітко визначений набір повноважень і доступу до відповідних функцій системи. Система повинна забезпечувати зручний механізм реєстрації, автентифікації та управління профілями користувачів, зберігати історію дій, а також дозволяти гнучко змінювати ролі відповідно до потреб редакції.

Невід’ємною частиною функціоналу має стати модуль подання та обробки рукописів. Авторам повинна бути надана можливість швидко й зручно завантажувати свої статті, додавати супровідні документи, редагувати метадані,

					КР.ІІЗ-4220.ІІЗ	Арк.
						18
Зм.	Арк.	№ докум.	Підпис	Дата		-

відстежувати статус розгляду своєї роботи. Важливо, щоб система інформувала авторів про кожен етап проходження рукопису – від первинної перевірки до остаточного рішення.

Для рецензентів необхідно забезпечити ефективний інструментарій для доступу до призначених рукописів, подання рецензій, збереження історії своїх відгуків, а також отримання сповіщень про нові призначення та результати розгляду. Рецензенти мають мати можливість працювати із шаблонами рецензій, а також взаємодіяти із редакцією щодо уточнення критеріїв оцінювання.

Редактори повинні отримати гнучкі засоби керування рукописами, призначення рецензентів, прийняття рішень, формування випусків журналу, а також ведення статистики та аналітики щодо роботи редакційної колегії. Важливою є можливість створення шаблонів листування, автоматичного надсилання повідомлень авторам і рецензентам на різних етапах розгляду рукописів.

Система має забезпечити автоматизацію процесу формування та публікації випусків журналу, експорту метаданих у міжнародні бази даних, присвоєння DOI, імпорт/експорт статей у форматах, сумісних із основними агрегаторами наукової інформації. Інтеграція із зовнішніми сервісами має бути максимально гнучкою, з можливістю підключення нових модулів без суттєвого втручання в основний код платформи.

Серед нефункціональних вимог особливе місце займає питання безпеки та захисту даних. Система повинна впроваджувати сучасні механізми шифрування, багаторівневої автентифікації, захищати персональні дані користувачів відповідно до міжнародних стандартів. Не менш важливим є забезпечення стабільної роботи платформи за високих навантажень, підтримка масштабування, резервного копіювання й оперативного відновлення даних у разі збоїв.

					КР.ІІЗ-4220.ІІЗ	Арк.
						19
Зм.	Арк.	№ докум.	Підпис	Дата		-

Вимоги до зручності користування також мають бути на високому рівні. Інтерфейс повинен бути інтуїтивно зрозумілим, адаптивним, локалізованим, із чіткою структурою розділів та швидким доступом до основних функцій. Важливо забезпечити підтримку роботи з різних пристроїв, включаючи мобільні, для підвищення доступності платформи.

Ще одним завданням є створення розвиненої системи сповіщень, яка дозволить оперативно інформувати всіх учасників процесу про зміни статусу рукописів, нові призначення, терміни виконання дій, а також забезпечить гнучке налаштування каналів надходження повідомлень (електронна пошта, push-сповіщення тощо).

Система повинна підтримувати ведення історії змін, логування всіх важливих подій, формування статистичних та аналітичних звітів для різних категорій користувачів. Це дозволить редакціям приймати обґрунтовані рішення щодо оптимізації власної роботи, а авторам і рецензентам – аналізувати власну публікаційну та рецензійну активність.

З огляду на вимоги відкритої науки, важливою вимогою є також підтримка відкритого доступу до публікацій, гнучкі механізми ліцензування, інтеграція з репозитаріями відкритих даних та забезпечення прозорості рецензійного процесу [4].

Отже, основною метою розробки є створення комплексної, сучасної науково-видавничої платформи, яка задовольнятиме вимоги редакцій наукових журналів, авторів, рецензентів та адміністраторів, забезпечить високу якість і безпеку наукової комунікації, інтеграцію із зовнішніми сервісами та відповідатиме найкращим міжнародним стандартам у сфері наукових публікацій.

					КР.ІІЗ-4220.ІІЗ	Арк.
						20
Зм.	Арк.	№ докум.	Підпис	Дата		-

2 ПРОЕКТУВАННЯ СИСТЕМИ

2.1 Вибір технологій

У сучасних умовах стрімкого розвитку інформаційних та комунікаційних технологій, коли цифровізація охоплює практично всі сфери суспільного життя, особливо зростають вимоги до розробки складних, багатофункціональних і надійних інформаційних систем. Створення інноваційної науково-видавничої платформи – це не просто оперативне написання програмного коду чи вибір популярних інструментів, а глибоко виважений процес, який передбачає ретельний аналіз технологічних тенденцій, потреб користувацької аудиторії, вимог до безпеки, масштабованості, інтеграцій та підтримки складних бізнес-процесів [7]. Саме тому правильний вибір технологічного стеку стає одним із ключових етапів усього життєвого циклу програмного продукту, що визначає його подальшу функціональність, надійність, зручність у супроводі та здатність до розвитку.

Одним із першочергових завдань при проектуванні платформи було забезпечення високого рівня модульності, щоб кожен функціональний блок системи – чи то реєстрація користувачів, подача і рецензування статей, керування журналами чи зберігання файлів – міг незалежно розвиватися, тестуватися та вдосконалюватися без негативного впливу на інші компоненти [16]. З цією метою як основу серверної частини обрано NestJS – сучасний прогресивний фреймворк, що поєднує переваги Node.js та строгої типізації TypeScript. NestJS надає розробникам інтуїтивно зрозумілу архітектуру, що базується на концепціях модулів, контролерів, сервісів і репозиторіїв, дозволяючи чітко розмежовувати відповідальність між різними частинами програми.

Завдяки застосуванню патернів проектування, декораторів, впровадженню інверсії залежностей, NestJS робить код більш структурованим, підтримуваним і

					КР.ІІЗ-4220.ІІЗ	Арк.
						21
Зм.	Арк.	№ докум.	Підпис	Дата		-

масштабованим. Це особливо важливо для командної розробки, коли над платформою працює декілька фахівців, і необхідно уникнути ситуацій, коли зміна одного модуля призводить до непередбачуваних збоїв в інших частинах системи. Крім того, використання TypeScript дозволяє мінімізувати кількість помилок на етапі компіляції, завдяки суворій типізації, що позитивно впливає на загальну якість продукту.

Ще одним вагомим аргументом на користь NestJS стала надзвичайно розвинена екосистема та активна міжнародна спільнота розробників, яка постійно поповнюється новими пакетами, розширеннями, бібліотеками для інтеграції з іншими популярними сервісами. Це дає змогу швидко і безпечно впроваджувати такі функції, як автентифікація, авторизація, логування, інтеграція з електронною поштою, моніторинг, робота з різними форматами файлів тощо. Регулярні оновлення та підтримка безпеки гарантують оперативне реагування на нові виклики та загрози, що є критично важливим для сучасних систем, які обробляють персональні дані та мають високі вимоги до захисту інформації.

Для зберігання структурованих даних було обрано одну з найпотужніших і найгнучкіших реляційних систем – PostgreSQL. Ця СКБД вже багато років утримує позиції лідера серед рішень для професійної розробки складних інформаційних систем. Її переваги – це не лише відкритий вихідний код і безкоштовне використання, а й активна спільнота, розвинена система розширень, підтримка складних транзакцій, високий рівень безпеки, можливість гнучкого індексування та роботи з великими обсягами інформації. Для науково-видавничої платформи це означає, що дані про користувачів, статті, журнали, рецензії, файли, історію дій та інші об'єкти зберігаються централізовано, із забезпеченням цілісності, логічних зв'язків та можливості виконання складних аналітичних вибірок.

					КР.ІІЗ-4220.ІІЗ	Арк.
						22
Зм.	Арк.	№ докум.	Підпис	Дата		-

Для максимально ефективної взаємодії між бізнес-логікою і базою даних використовується ORM-бібліотека TypeORM, яка дозволяє розробникам працювати із сутностями на рівні об'єктно-орієнтованого програмування. Усі таблиці, зв'язки, обмеження описуються у вигляді класів і декораторів, що значно спрощує розробку, рефакторинг і подальші міграції. Актуалізація структури БД, додавання нових атрибутів чи сутностей виконується через систему міграцій, що дозволяє уникнути втрати даних і мінімізувати людський фактор під час оновлення платформи. Додатково, TypeORM забезпечує прозоре керування зовнішніми ключами, індексами, каскадними видаленнями, що критично важливо для підтримки складної предметної області.

Безпека персональних даних та контроль доступу до ресурсів – один із наріжних каменів будь-якої сучасної ІТ-системи. У платформі реалізовано захищену аутентифікацію користувачів за допомогою JWT – технології, що дозволяє формувати цифровий підпис із вказанням ролей, ідентифікаторів, строку дії та інших параметрів. Це надає змогу будувати гнучкі політики доступу, впроваджувати middleware для захисту маршрутів, а також швидко масштабувати систему за потреби. Для збереження паролів застосовується сучасний алгоритм шифрування bcrypt, що значно знижує ризик компрометації навіть у разі витоку даних.

Особлива роль у платформі відведена організації роботи з файлами, адже науково-видавнича система передбачає збереження великої кількості рукописів, рецензій, супровідних документів, ілюстрацій тощо. Для цього використовуються як стандартні інструменти Node.js, так і спеціалізовані модулі NestJS, які дозволяють зберігати файли як у локальному файловому сховищі, так і в хмарних середовищах (Amazon S3, Google Cloud Storage). Це забезпечує гнучкість, масштабованість, організацію резервного копіювання, швидке відновлення у разі збоїв, а також контроль прав доступу до кожного окремого файлу.

					КР.ІІЗ-4220.ІІЗ	Арк.
						23
Зм.	Арк.	№ докум.	Підпис	Дата		-

Інтерфейсна частина системи (frontend) проектується із використанням сучасних JavaScript-фреймворків – React або Angular. Вони дозволяють створювати швидкі, адаптивні, інтуїтивно зрозумілі інтерфейси користувача, що забезпечують комфортну роботу як для авторів, так і для рецензентів, редакторів і адміністраторів. Оскільки серверна частина реалізує RESTful API, платформа легко масштабується та інтегрується з будь-якими сучасними веб- і мобільними клієнтами, що забезпечує її перспективність та універсальність [15].

Для підтримки високого рівня якості коду, автоматизації розгортання, тестування й оновлення системи, у проєкті впроваджено практики CI/CD. GitHub Actions використовується для автоматизації процесів тестування, збирання, деплоюменту, що дозволяє швидко виявляти помилки, контролювати якість коду, оперативно впроваджувати оновлення без простоїв. Контейнеризація із використанням Docker забезпечує ізолюваність середовищ, спрощує перенесення системи на різні сервери, мінімізує залежність від особливостей конкретної інфраструктури.

Моніторинг та логування подій – ще один невід’ємний компонент сучасної ІТ-системи. Для цього у платформі застосовуються бібліотеки Winston або Pino, а також сервіси Prometheus і Grafana, які дозволяють у режимі реального часу відстежувати стан системи, навантаження, кількість користувачів, статистику запитів, оперативно реагувати на збої та забезпечувати високу доступність.

Таким чином, обраний технологічний стек забезпечує оптимальний баланс між інноваційністю, стабільністю, безпекою, швидкістю розробки та відкритістю до інтеграцій. Це гарантує не лише відповідність сучасним міжнародним стандартам у галузі розробки інформаційних систем, а й закладає надійну основу для подальшого розвитку, масштабування та впровадження нових функцій науково-видавничої платформи.

					КР.ІІЗ-4220.ІІЗ	Арк.
						24
Зм.	Арк.	№ докум.	Підпис	Дата		-

2.2 Архітектура системи

2.2.1 Загальна архітектура системи

Розроблена система базується на класичній трирівневій клієнт-серверній архітектурі із чітким розмежуванням обов'язків між клієнтською частиною (front-end), серверною частиною (back-end) та базою даних. Такий підхід дозволяє не тільки підвищити гнучкість і масштабованість рішення, а й істотно спрощує подальший супровід, тестування та впровадження нових функцій у майбутньому.

Загалом проект виглядатиме так:

- Клієнтська частина системи – виконує роль інтерфейсу для кінцевих користувачів, відповідає за відображення інформації, взаємодію із користувачем та передачу даних до серверної частини через API. Реалізація виконана як Single Page Application (SPA) із використанням сучасної компонентної архітектури (наприклад, React), що забезпечує не лише високу швидкодію, а й інтуїтивно зрозумілий, динамічний і привабливий інтерфейс [17]. SPA дозволяє здійснювати навігацію між сторінками без повного перезавантаження, що позитивно впливає на досвід користувача, а також відкриває можливості для реалізації адаптивного дизайну й інтерактивних елементів.
- Серверна частина – є ядром системи, оскільки на ній концентрується вся бізнес-логіка, обробка запитів від клієнта, взаємодія з базою даних, перевірка прав доступу, обробка файлів, а також інтеграція із зовнішніми сервісами (наприклад, ORCID, DOI, email-повідомлення, сторонні сервіси зберігання тощо). Реалізація здійснюється із використанням фреймворку NestJS, що базується на Node.js і підтримує модульний підхід, інверсію контролю, декоратори, сувору типізацію коду (TypeScript), що, в свою чергу, підвищує надійність,

					КР.ІІЗ-4220.ІІЗ	Арк.
						25
Зм.	Арк.	№ докум.	Підпис	Дата		-

підтримуваність і розширюваність усієї серверної логіки. NestJS дозволяє організувати код за принципами SOLID, забезпечує підтримку middleware, guards, filters, pipes – усіх необхідних для сучасної enterprise-розробки механізмів [21].

- Сховище даних у системі – представлено реляційною СУБД PostgreSQL, що дозволяє зберігати структуровану інформацію про всі основні об’єкти доменної області: користувачів, журнали, випуски, статті, рецензії, файли, сповіщення тощо. PostgreSQL обрано завдяки її відкритості, потужним функціям для складних зв’язків між даними, високій продуктивності, розвинутих механізмах транзакційності, гнучкій роботі з типами даних, а також активній підтримці спільноти.

Окрім основних компонентів, архітектура передбачає наявність низки допоміжних сервісів, зокрема:

- Сервіс автентифікації та авторизації – реалізує механізми реєстрації користувачів, перевірки облікових даних, генерування та перевірки JWT-токенів для управління сесіями, а також хешування паролів із використанням bcrypt.
- Сервіс зберігання файлів – забезпечує прийом, перевірку, зберігання та видачу файлів, пов’язаних із статтями, рецензіями, журналами; метадані файлів зберігаються у БД, самі файли – у файловій системі чи хмарному сховищі.

2.2.2 Архітектура клієнтської частини

Розробка клієнтської частини базується на використанні сучасного підходу SPA, який, на відміну від класичних багатосторінкових рішень, дозволяє значно покращити взаємодію користувача із системою, забезпечити швидкість реакції, плавність переходів, відсутність надмірних завантажень сторінки. Компонентна архітектура (наприклад, React) сприяє гнучкості, легкості повторного використання коду, а також спрощує підтримку та масштабування інтерфейсу.

До основних модулів клієнтської частини належать:

					КР.ІІЗ-4220.ІІЗ	Арк.
						26
Зм.	Арк.	№ докум.	Підпис	Дата		-

- Модуль автентифікації (Authentication Module) – забезпечує реєстрацію нових користувачів, вхід до системи, відновлення та зміну пароля. Відповідає за безпечне зберігання та передачу токенів автентифікації (JWT), перевірку прав доступу та автоматичне оновлення сесії користувача. Також включає механізми підтвердження email та захисту від несанкціонованого доступу.
- Модуль профілю користувача (User Profile Module) – дозволяє користувачам переглядати та редагувати власну інформацію: ПІБ, контактні дані, науковий ступінь, профіль ORCID, пароль та інші персональні налаштування. Передбачає можливість зміни фото профілю, відображення історії дій та персональних сповіщень.
- Модуль подання рукопису (Manuscript Submission Module) – реалізує функціонал створення нової статті/рукопису з введенням необхідних метаданих (заголовки, анотація, ключові слова, тип, журнал, список авторів) та завантаженням основних файлів. Дозволяє редагувати поданий рукопис до моменту його остаточного відправлення на розгляд.
- Модуль управління рукописами (Manuscript Management Module) – Надає користувачу можливість переглядати список своїх статей, відслідковувати статус кожної (чернетка, на розгляді, рецензування, прийнято, опубліковано, відхилено), редагувати або видаляти чернетки, отримувати інформацію про терміни розгляду та історію змін по кожному рукопису.
- Модуль керування рецензіями (Review Management Module) – дозволяє рецензентам переглядати призначені їм рукописи, заповнювати форму рецензії з коментарями та оцінками, завантажувати супровідні файли. Передбачає зручне спілкування між рецензентом і редактором, а також можливість бачити історію власних рецензій.

- Модуль керування журналами (Journal Management Module) – реалізує адміністрування журналів: створення та редагування метаданих, налаштування політик, призначення редакторів, управління випусками (створення, публікація, архівація), перегляд списку опублікованих статей, налаштування інтеграції з зовнішніми сервісами (Crossref, DOI).
- Модуль сповіщень (Notification Module) – відповідає за відображення всіх важливих змін у системі: нові призначення на рецензію, зміна статусу рукопису, повідомлення від редакції, оновлення профілю чи пароля. Дозволяє користувачу переглядати історію сповіщень, відмічати їх як прочитані або видаляти.
- Модуль адміністрування (Admin Module) – доступний лише адміністраторам і редакторам. Дозволяє управляти користувачами, їхніми ролями та правами доступу, переглядати та редагувати глобальні налаштування системи, контролювати журнали, випуски, рецензентів, створювати звіти та переглядати логи.
- Загальні/спільні компоненти (Shared/Common Components) – містять елементи, що використовуються у всіх модулях: навігаційне меню, шапка, футер, таблиці, фільтри, модальні вікна, форми, повідомлення про помилки, завантажувачі файлів, валідація даних та інше.

Клієнтська частина також забезпечує зберігання токенів доступу (JWT) локально (наприклад, у localStorage), що дозволяє зберігати сесію користувача навіть після оновлення сторінки.

2.2.3 Архітектура серверної частини

Серверна частина системи реалізована на основі NestJS – сучасного фреймворку для Node.js, який поєднує у собі кращі практики класичних enterprise-рішень із гнучкістю екосистеми JavaScript/TypeScript.

Архітектура серверу організована за багат шаровим принципом:

					КР.ІІЗ-4220.ІІЗ	Арк.
						28
Зм.	Арк.	№ докум.	Підпис	Дата		-

- Шар контролерів (Controllers) – відповідає за прийом HTTP-запитів, виконання первинної валідації, передачу даних у відповідні сервіси та формування структурованої відповіді у форматі JSON. Для кожної сутності (користувачі, статті, журнали, рецензії тощо) реалізовано окремий контролер.
- Шар сервісів (Services) – містить основну бізнес-логіку системи. Саме тут реалізуються сценарії обробки даних, перевірка прав доступу, взаємодія з іншими сервісами, підготовка відповідей для контролерів.
- Шар доступу до даних (Repositories/Entities) – відповідає за взаємодію з базою даних через ORM TypeORM. Кожна сутність описується окремим класом (entity), де вказуються всі атрибути, типи даних, зв'язки з іншими сутностями, правила збереження та вибірки.
- Шар інтеграції із зовнішніми сервісами – містить модулі для взаємодії із зовнішніми API (електронна пошта, ORCID, DOI, файлові сховища).
- Шар конфігурації, логування та утиліт – включає механізми для зберігання та читання конфігурацій оточення, організації логування подій, обробки помилок, глобальних фільтрів, middleware.

Усе це дозволяє розділити відповідальність, зробити код легким для тестування, розширення та супроводу, а також забезпечити високий рівень безпеки та надійності системи.

2.2.4 Взаємодія компонентів системи

Комунікація між клієнтом і сервером здійснюється через стандартизований REST API, що гарантує уніфікований підхід до обміну даними [18]. Усі запити та відповіді формуються у форматі JSON, що спрощує інтеграцію з іншими системами та забезпечує швидкість передачі даних.

Основні сценарії взаємодії охоплюють:

					КР.ІІЗ-4220.ІІЗ	Арк.
						29
Зм.	Арк.	№ докум.	Підпис	Дата		-

- Реєстрацію, автентифікацію та авторизацію користувачів із використанням JWT.
- Створення, редагування, перегляд, публікацію та видалення статей.
- Призначення рецензентів, створення та збереження рецензій, комунікацію між авторами, редакторами та рецензентами.
- Управління журналами, випусками, редакторськими колегіями.
- Завантаження, зберігання та отримання файлів, пов'язаних із статтями, рецензіями або журналами.
- Надсилання та отримання сповіщень, які інформують користувачів про важливі події у системі.

Вся взаємодія здійснюється із суворим контролем прав доступу, що забезпечує захист даних і мінімізує ризики несанкціонованого доступу.

2.2.5 Безпека та масштабованість

Безпека є одним з ключових пріоритетів у проектуванні та реалізації системи. Всі критичні дані (паролі, токени) зберігаються у захищеному вигляді. Застосовується багаторівнева валідація даних, а також захист від поширених веб-вразливостей (SQL-ін'єкцій, XSS, CSRF). Реалізовано розмежування прав доступу для різних типів користувачів, що підвищує рівень захисту та дозволяє чітко контролювати можливості кожної ролі.

Для забезпечення масштабованості система спроектована таким чином, щоб дозвляти горизонтальне та вертикальне масштабування. Рознесення функціональних модулів, ізольованість front-end і back-end, застосування контейнеризації (Docker), підтримка CI/CD – усе це забезпечує можливість швидкого розгортання нових інстансів, балансування навантаження, автоматичного оновлення, резервного копіювання та відновлення даних.

					КР.ІІЗ-4220.ІІЗ	Арк.
						30
Зм.	Арк.	№ докум.	Підпис	Дата		-

2.3 Проектування схеми бази даних

У процесі створення науково-видавничої системи надзвичайно важливим етапом є ретельне проектування схеми бази даних, що є фундаментом для забезпечення надійного, цілісного та ефективного зберігання інформації. Саме від структури бази даних залежать не лише швидкодія платформи та простота реалізації бізнес-логіки, а й здатність системи до масштабування, інтеграції із зовнішніми сервісами, а також безпека і цілісність усіх даних.

Проектування схеми БД розпочинається з аналізу предметної області та виявлення ключових сутностей, які відображають основні об'єкти у системі та їхню взаємодію. До таких сутностей належать: користувачі, журнали, випуски, статті, рецензії, файли, сповіщення, а також допоміжні таблиці, що дозволяють реалізувати ролі, статуси, історію змін, інтеграцію з зовнішніми джерелами тощо.

Таблиці бази даних:

- users – зберігає інформацію про користувачів системи (автори, рецензенти, редактори, адміністратори).
- journals – містить дані про наукові журнали, які адмініструються на платформі.
- issues – описує випуски журналів (номер, рік, дата публікації тощо).
- articles – зберігає основну інформацію про наукові статті (метадані, статус, зв'язки з авторами, журналом і випуском).
- reviews – містить рецензії на статті, оцінки й коментарі рецензентів.
- article_files – зберігає метадані всіх файлів, пов'язаних зі статтями (основний текст, додаткові матеріали).
- notifications – фіксує сповіщення для користувачів про важливі події в системі.
- article_authors – допоміжна таблиця для зв'язку “багато-до-багатьох” між статтями й їхніми авторами.

					КР.ІІЗ-4220.ІІЗ	Арк.
						31
Зм.	Арк.	№ докум.	Підпис	Дата		-

- journal_editors – допоміжна таблиця для зв'язку журналів та їх редакторів.

Таблиця користувачів (users) є однією з центральних у структурі бази даних науково-видавничої системи (див. рисунок 2.1). Вона зберігає всю основну інформацію про зареєстрованих користувачів, забезпечуючи ідентифікацію, авторизацію, а також персоналізацію взаємодії з платформою. Завдяки багатому набору атрибутів ця сутність дозволяє реалізувати розмежування прав, підтримку наукового профілю, зберігання біографічних даних та інтеграцію із зовнішніми сервісами. Роль таблиці у системі полягає у ідентифікації та автентифікації, що забезпечує унікальність користувачів, а також безпечний доступ до системи. Також розмежування прав і ролей, завдяки атрибуту roles система може призначати різні рівні доступу – від звичайного автора до адміністратора. Наукова та організаційна ідентифікація поля institution, academicDegree, orcidId та bio дають змогу створити повноцінний науковий профіль. Зручність взаємодії та персоналізація: firstName, lastName, profilePicture та bio створюють комфортний інтерфейс для користувачів усіх ролей [10].

users	
firstName	varchar
lastName	varchar
email	
password	
roles	users_roles
institution	
orcidId	
academicDegree	
bio	varchar
profilePicture	

Рисунок 2.1 – Таблиця users

Таблиця journals є фундаментальною для роботи всієї системи, оскільки саме вона визначає структуру та характеристики наукових журналів, які адмініструються на платформі (див. рисунок 2.2). Вона містить повний набір важливих параметрів, що дають змогу створити окремий профіль для кожного журналу, описати його особливості, політики, контактні дані, видавця, а також встановити правила взаємодії з авторами, рецензентами й читачами. Завдяки цій таблиці забезпечується унікальність кожного журналу в системі, підтримується багатомовність, тематичне розмаїття та відповідність сучасним міжнародним стандартам наукової комунікації (наприклад, наявність ISSN, eISSN, open access, APC) [8].

Таблиця дає можливість адміністраторам системи та редакторам легко додавати нові журнали, редагувати їхні властивості, архівувати неактуальні видання або призупиняти їхню діяльність. Вона також слугує точкою інтеграції із зовнішніми сервісами, агрегаторами, наукометричними базами та реєстрами, оскільки містить всі атрибути, необхідні для автоматизованого експорту даних або синхронізації з іншими інформаційними платформами.

До того ж, таблиця journals є базою для всіх пов'язаних процесів: формування випусків (issues), публікації статей (articles), призначення редакторів (journal_editors), ведення контактів з авторами та рецензентами. Вона дозволяє структурувати журнали за тематикою, мовами, політикою відкритого доступу, що значно полегшує пошук необхідної інформації для користувачів різних ролей. Окрема увага приділяється питанням прозорості – можливість зберігати детальні описи, інструкції для авторів, опис процесу рецензування, контактні дані редакції та іншу інформацію забезпечує довіру до платформи та підвищує її привабливість для наукової спільноти.

					КР.ІІЗ-4220.ІІЗ	Арк.
						33
Зм.	Арк.	№ докум.	Підпис	Дата		-

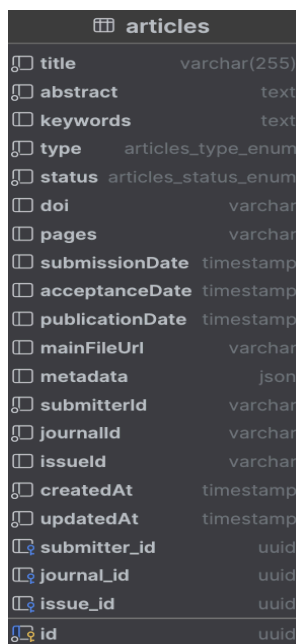
journals	
title	varchar(255)
slug	varchar(255)
description	text
coverImage	varchar
issn	varchar
eissn	varchar
publisher	varchar
languages	text
keywords	text
aimsAndScope	text
status	journals_status_enum
submissionGuidelines	text
peerReviewProcess	text
publicationFrequency	varchar
isOpenAccess	boolean
apcFee	integer
contactInformation	json
createdAt	timestamp
updatedAt	timestamp
id	uuid

Рисунок 2.2 – Таблиця journals

Таблиця articles є центральною частиною системи, оскільки саме вона містить всю основну інформацію про наукові статті, що надходять і обробляються на платформі (див. рисунок 2.3). Через цю таблицю реалізується повний життєвий цикл статті: від моменту подання рукопису автором, проходження етапів рецензування та прийняття рішення редакцією, до остаточної публікації у відповідному випуску журналу. Вона акумулює ключові метадані кожної статті – такі як назва, анотація, ключові слова, тип, статус, DOI, кількість сторінок, а також фіксує дати основних етапів опрацювання (подання, прийняття, публікації). Окремі поля дозволяють зберігати посилання на основний файл рукопису, додаткову структуровану інформацію для інтеграції з зовнішніми базами, і забезпечують зв'язки з журналом, випуском та користувачем, який подав статтю [9].

Завдяки цій таблиці система може організовувати ефективний процес рецензування, забезпечувати прозорість і контроль над статусом кожної статті, автоматизувати формування випусків журналу, вести облік авторства та здійснювати пошук і фільтрацію матеріалів за різними критеріями. Вона також

дозволяє інтегруватися з міжнародними системами наукової ідентифікації та забезпечує гнучкість для подальшого розширення функціоналу платформи.



articles	
title	varchar(255)
abstract	text
keywords	text
type	articles_type_enum
status	articles_status_enum
doi	varchar
pages	varchar
submissionDate	timestamp
acceptanceDate	timestamp
publicationDate	timestamp
mainFileUrl	varchar
metadata	json
submitterId	varchar
journalId	varchar
issueId	varchar
createdAt	timestamp
updatedAt	timestamp
submitter_id	uuid
journal_id	uuid
issue_id	uuid
id	uuid

Рисунок 2.3 – Таблиця articles

Таблиця issues відповідає за зберігання й організацію випусків наукових журналів у системі. Вона містить інформацію про кожен окремий випуск, зокрема його назву, опис, обкладинку, номер тому, номер випуску, дату публікації, статус, DOI, а також зв'язок із конкретним журналом (див. рисунок 2.4).

За допомогою цієї таблиці можна структурувати всі статті, які входять до певного випуску, забезпечити прозорість формування архіву публікацій та автоматизувати підготовку нових номерів. Вона дозволяє зберігати історію змін для кожного випуску, а також підтримує інтеграцію з міжнародними системами через наявність поля DOI. Таким чином, таблиця issues є основною для адміністрування номерів журналів і забезпечує логічне групування статей у межах випусків, що важливо як для редакторів, так і для користувачів платформи.

issues	
title	varchar(255)
description	text
coverImage	varchar
volume	integer
number	integer
publicationDate	date
status	issues_status_enum
doi	varchar
journalId	varchar
createdAt	timestamp
updatedAt	timestamp
journal_id	uuid
id	uuid

Рисунок 2.4 – Таблиця issues

Таблиця reviews відповідає за зберігання та організацію процесу рецензування наукових статей у системі (див. рисунок 2.5). Вона містить усі необхідні дані, щоб відображати повний життєвий цикл рецензії: від призначення рецензента до остаточного рішення щодо статті. У цій таблиці фіксуються зв'язки із статтею, рецензентом, а також користувачем, який призначив рецензію. Зберігаються статус рецензії, рекомендація рецензента, коментарі як для автора, так і для редактора, а також можливість додати файл із рецензією. Окремі поля дозволяють деталізувати оцінювання рукопису за кількома критеріями (якість, оригінальність, релевантність, ясність), забезпечити анонімність рецензування, а також контролювати строки виконання через дати дедлайну й завершення.

Таблиця також містить часові мітки створення та оновлення, що дозволяє відстежувати динаміку рецензування та формувати повну історію взаємодії для кожної статті. У підсумку, reviews є ключовою для автоматизації процесу рецензування, підвищення прозорості, об'єктивності та якості редакційної роботи платформи.

					КР.ІІЗ-4220.ІІЗ	Арк.
						36
Зм.	Арк.	№ докум.	Підпис	Дата		-

reviews	
articleId	varchar
reviewerId	varchar
assignedById	varchar
status	reviews_status_enum
recommendation	reviews_recommendation_enum
commentsToAuthor	text
commentsToEditor	text
fileUrl	varchar
qualityScore	integer
originalityScore	integer
relevanceScore	integer
clarityScore	integer
anonymous	boolean
dueDate	timestamp
completionDate	timestamp
createdAt	timestamp
updatedAt	timestamp
article_id	uuid
reviewer_id	uuid
assigned_by_id	uuid
id	uuid

Рисунок 2.5 – Таблица reviews

Таблица `article_files` відповідає за зберігання всієї інформації про файли, які прикріплені до наукових статей у системі (див. рисунок 2.6). Вона містить назву файлу, шлях до нього, розмір, тип і MIME-тип, а також додаткові характеристики, такі як опис файлу, номер версії та дату завантаження або оновлення. Кожен файл пов'язаний із певною статтею, що дає змогу зберігати не лише основний рукопис, а й додаткові матеріали – ілюстрації, таблиці, презентації, супровідні документи тощо.

Завдяки цій таблиці система забезпечує зручну структуру для організації всіх файлів, пов'язаних зі статтею, підтримує версіонування та дозволяє редакції або авторам ефективно управляти всіма супровідними матеріалами на кожному етапі публікаційного процесу.

article_files	
filename	varchar(255)
fileUrl	varchar
fileSize	integer
mimeType	varchar(100)
fileType	article_files_filetype_enum
description	varchar
version	integer
articleId	varchar
createdAt	timestamp
updatedAt	timestamp
article_id	uuid
id	uuid

Рисунок 2.6 – Таблиця article_files

Таблиця article_authors слугує проміжною ланкою для організації зв'язку між статтями та їхніми авторами у системі. Вона реалізує зв'язок «багато-до-багатьох», оскільки одна стаття може мати кількох авторів, і один користувач може бути автором багатьох статей (див. рисунок 2.7).

Завдяки цій таблиці система точно відображає авторський склад кожної публікації, дозволяє належним чином ідентифікувати та відображати інформацію про авторів у профайлах статей, а також забезпечує гнучкість у пошуку й фільтрації наукових робіт за конкретними користувачами.

article_authors	
article_id	uuid
user_id	uuid

Рисунок 2.7 – Таблиця article_authors

Таблиця journal_editors забезпечує зберігання інформації про зв'язки між користувачами та науковими журналами на платформі в контексті виконання ними редакторських функцій (див. рисунок 2.8). Вона реалізує зв'язок «багато-

до-багатьох»: один журнал може мати кількох редакторів, і один користувач може бути редактором для різних журналів.

Завдяки цій таблиці система дозволяє ефективно управляти редакційними колективами, призначати і змінювати склад редакторів для кожного журналу, а також організовувати розмежування прав доступу та відповідальності у процесі редакційного менеджменту.

journal_editors	
journal_id	uuid
user_id	uuid

Рисунок 2.8 – Таблиця journal_editors

Таблиця notifications призначена для зберігання системних сповіщень, які надсилаються користувачам платформи (див. рисунок 2.9). Вона містить заголовок, текст повідомлення, тип сповіщення, посилання для переходу до відповідної події, а також додаткові метадані у форматі JSON. Для кожного сповіщення фіксується, чи було воно прочитане користувачем, чи надіслано на електронну пошту, а також дати створення і оновлення. Таблиця пов'язана з конкретним користувачем, що дозволяє формувати індивідуальні стрічки повідомлень, підвищуючи інформованість і залучення користувачів у процесах платформи.

📦 notifications	
🔑 userId	varchar
🔑 title	varchar(255)
🔑 message	text
🔑 type	notifications_type_enum
🔑 linkUrl	varchar
🔑 metadata	json
🔑 isRead	boolean
🔑 isEmailed	boolean
🔑 createdAt	timestamp
🔑 updatedAt	timestamp
🔑 user_id	uuid
🔑 id	uuid

Рисунок 2.9 – таблиця notifications

Для забезпечення прозорості і контролю дій у системі реалізується “Історія змін”, де зберігається інформація про всі суттєві операції: дата, тип події, суб’єкт, об’єкт змін. Це дозволяє проводити аудит, аналізувати процеси та відновлювати історію за необхідності.

Всі сутності пов’язані між собою через зовнішні ключі. Так, стаття має посилання на журнал, випуск і авторів; рецензія – на статтю й рецензента; файл – на статтю або рецензію. Це забезпечує логічну цілісність, дозволяє будувати складні вибірки, швидко фільтрувати дані та уникати дублювання інформації.

Візуально типи зв’язків між таблицями показано в ERD-діаграмі бази даних (див. додаток А).

Для підвищення продуктивності використовуються індекси на основних полях пошуку (електронна пошта, DOI, назви статей, дати подання, статуси). Механізми міграцій, що реалізуються через TypeORM, забезпечують надійне оновлення структури БД без втрати даних. Завдяки такому підходу схема бази даних стає гнучкою, масштабованою та готовою до подальшого розвитку, дозволяючи ефективно реалізовувати як поточні, так і майбутні функції платформи. [9]

2.4 Визначення ролей користувачів та бізнес-процесів

Організація ефективної взаємодії між різними учасниками науково-видавничого процесу неможлива без чіткого визначення ролей користувачів і детального опису бізнес-процесів [19]. Це забезпечує не лише прозорість і контрольованість усіх етапів життєвого циклу статті, а й дозволяє автоматизувати складні сценарії, мінімізувати ризики помилок та оптимізувати роботу редакційної колегії.

У системі виділяється низка ключових ролей, кожна з яких має власний набір прав, сценаріїв взаємодії та відповідальності.

Автор – користувач, який ініціює публікаційний процес, подає рукописи, заповнює метадані, завантажує супровідні документи, отримує повідомлення про перебіг розгляду, взаємодіє з редакцією щодо доопрацювань. Для авторів реалізовано інтуїтивний інтерфейс подачі рукопису, що забезпечує покрокове введення інформації, можливість редагування поданих даних до певного етапу, а також відстеження статусу та історії змін по своїх статтях.

Рецензент – фахівець, який після запрошення редакції приймає або відхиляє пропозицію щодо рецензування, отримує доступ до анонімізованого рукопису, формує відгук за структурованою формою, залишає коментарі для авторів і редакторів, може додавати додаткові файли (наприклад, відмічені рукописи). Система автоматично контролює строки виконання рецензій, надсилає нагадування, дозволяє рецензенту зберігати чернетки відгуків. Для підтримки різних моделей рецензування реалізовано механізми анонімізації та контролю видимості інформації.

Редактор – ключова фігура, що координує процес розгляду рукописів, призначає рецензентів, аналізує отримані відгуки, приймає рішення щодо статті, комунікує з авторами, стежить за дотриманням строків, формує випуски журналу. Редактор має розширені права щодо перегляду аналітики, історії змін,

					КР.ІІЗ-4220.ІІЗ	Арк.
						41
Зм.	Арк.	№ докум.	Підпис	Дата		-

керування шаблонами листування, а також може делегувати окремі повноваження.

Адміністратор – користувач із найширшими правами, що відповідає за створення й адміністрування журналів та випусків, керування ролями, налаштування інтеграцій, аудит подій, аналіз статистики роботи системи. Адміністратор забезпечує безперервність роботи, безпеку платформи, резервне копіювання, супровід документації, а також може налаштовувати бізнес-процеси для окремих журналів [8, 21].

Бізнес-процеси системи побудовані навколо життєвого циклу наукової статті. Початковий етап включає реєстрацію автора й подачу рукопису, після чого проводиться автоматична перевірка на відповідність формальним вимогам. Далі технічний секретар або редактор здійснює первинний контроль, після якого рукопис надходить на розгляд редакційної колегії. Вибрані рецензенти отримують запрошення, виконують аналіз і формують відгуки. Редактор, на основі рецензій, приймає рішення щодо статті й повідомляє автора про результати.

У разі необхідності доопрацювання автор отримує коментарі, вносить зміни, повторно подає оновлену версію рукопису. Всі дії супроводжуються автоматичними сповіщеннями й фіксуються в історії змін. Після остаточного прийняття статті редактор або технічний секретар готує матеріал до публікації, формує випуск, присвоює DOI, експортує дані до міжнародних індексів. Адміністратор здійснює аудит, контролює системні процеси, забезпечує стабільність роботи.

Гнучкість бізнес-процесів досягається завдяки можливості налаштування кількості рецензій, типу рецензування, строків виконання, правил розподілу рукописів для кожного журналу окремо. Це дозволяє платформі залишатися універсальною, масштабованою та зручною для різних наукових спільнот.

					КР.ІІЗ-4220.ІІЗ	Арк.
						42
Зм.	Арк.	№ докум.	Підпис	Дата		-

2.5 Опис API (основні ендпоінти, маршрути, методи)

2.5.1 Філософія побудови API

Розробка API базується на сучасних принципах архітектури REST (Representational State Transfer), що передбачає використання чітких, стандартизованих HTTP-методів для організації взаємодії з ресурсами.

До основних HTTP-методів, які використовуються у системі, належать:

- GET – для отримання інформації про ресурси (наприклад, перегляд списку статей, отримання даних про користувача чи журнал);
- POST – для створення нових записів (наприклад, подання рукопису, реєстрація нового користувача, додавання рецензії);
- PUT/PATCH – для оновлення існуючих ресурсів (редагування профілю, оновлення статусу статті, редагування випуску тощо);
- DELETE – для видалення ресурсів (видалення файлів, рукописів, сповіщень).

Кожен тип сутності (користувачі, журнали, статті, рецензії, файли, сповіщення) має власний набір маршрутів (endpoint-ів), які логічно групуються за функціоналом. Такий підхід забезпечує стандартизацію обробки запитів, спрощує інтеграцію нових клієнтів, а також дозволяє легко документувати й тестувати API [7].

2.5.2 Організація маршрутів та ресурсів

Архітектура API проектується таким чином, щоб кожна сутність системи могла бути повноцінно керованою через відповідні маршрути. Це означає, що будь-який об'єкт (наприклад, стаття чи користувач) може бути створений, змінений, видалений або отриманий через відповідний endpoint (див. додаток Б).

					КР.ІІЗ-4220.ІІЗ	Арк.
						43
Зм.	Арк.	№ докум.	Підпис	Дата		-

Всі маршрути організовано за ключовими напрямками, які відповідають структурі предметної області:

Користувачі та автентифікація:

- Передбачено набір ендпоїнтів для реєстрації, входу в систему, отримання та оновлення особистого профілю, відновлення пароля, а також для отримання інформації про інших користувачів із врахуванням їхніх ролей. Усі ці операції захищені за допомогою JWT-автентифікації, middleware та guard-ів, які перевіряють рівень доступу.

Журнали та випуски:

- Реалізовано маршрути для отримання списку журналів, деталізованої інформації про кожен журнал, створення нових журналів (тільки адміністраторами), редагування параметрів, видалення журналів, а також для керування випусками (отримання списків, створення, редагування, видалення).

Статті:

- API забезпечує можливість пошуку й фільтрації статей за різними критеріями (статус, автор, журнал, ключові слова), подання нових рукописів, отримання детальної інформації, редагування метаданих чи файлів, перегляд історії змін, а також видалення статей із контролем прав доступу для різних ролей [19].

Рецензії:

- Для кожної статті можна отримати перелік рецензій, додати нову рецензію (рецензентом), переглянути, оновити чи видалити рецензію (залежно від ролі користувача та статусу процесу).

Файли:

- Передбачено маршрути для завантаження рукописів, рецензій, супровідних документів, отримання списку файлів, завантаження та видалення непотрібних документів із контролем прав доступу.

					КР.ІІЗ-4220.ІІЗ	Арк.
						44
Зм.	Арк.	№ докум.	Підпис	Дата		-

Сповіщення:

- Реалізовано механізми для отримання списку сповіщень, оновлення статусу (прочитано/непрочитано), видалення повідомлень, що дозволяє оптимізувати комунікацію та інформування користувачів.

Аналітика та аудит:

- Для адміністраторів відкрито спеціальні маршрути для перегляду статистики по користувачах, статтях, журналах, випусках, а також для аудиту ключових подій і перегляду логів системи.

2.5.3 Безпека, контроль доступу та валідація

Окрему увагу в архітектурі API приділено питанням безпеки та контролю доступу. Система використовує JWT (JSON Web Token) для аутентифікації користувачів: після входу користувач отримує унікальний токен, з яким здійснює усі запити до захищених маршрутів. Кожен запит обробляється через middleware та guard-и, які перевіряють валідність токена, роль користувача й права на виконання певної операції. Це дозволяє забезпечити багаторівневий захист, запобігати несанкціонованому доступу, а також реалізувати диференційовані права для авторів, рецензентів, редакторів, адміністраторів [17].

Для кожного маршруту передбачено використання DTO (Data Transfer Object) для валідації вхідних даних. Це дозволяє максимально рано виявляти помилки, забезпечувати цілісність і коректність даних, які потрапляють у систему, та повертати користувачам інформативні повідомлення про причину відмови у виконанні запиту.

2.5.4 Документування та інтерактивність

Ще одним важливим аспектом сучасного API є його якісне і повне документування. Для цього використовується інтеграція з інструментами

					КР.ІІЗ-4220.ІІЗ	Арк.
						45
Зм.	Арк.	№ докум.	Підпис	Дата		-

Swagger (OpenAPI), яка дозволяє автоматично генерувати інтерактивну документацію, де всі маршрути API, формати запитів та відповідей, типи даних, опис помилок і приклади наведено у зручній, зрозумілій для розробників формі. Це значно полегшує процес інтеграції зовнішніх клієнтів, мобільних застосунків, сторонніх сервісів, а також прискорює розробку фронтенду.

2.5.5 Версіонування API та backward compatibility

Архітектура API передбачає підтримку версіонування (наприклад, всі основні маршрути знаходяться у просторі /api/v1/...). Це дозволяє в майбутньому розвивати систему, додавати нові можливості, змінювати структуру даних або логіку роботи окремих функцій без порушення сумісності з уже існуючими клієнтами. Такий підхід забезпечує стабільність роботи, знижує ризики під час оновлення, дозволяє підтримувати одночасно декілька версій інтерфейсу [18].

2.5.6 Інтеграція з зовнішніми сервісами

API платформи розроблено з урахуванням необхідності глибокої інтеграції з міжнародними науковими сервісами та інфраструктурою, такими як DOI (Digital Object Identifier), ORCID, Crossref, антиплагіатні системи тощо. Для цього передбачено окремі маршрути та механізми авторизації, які гарантують захищену взаємодію, дотримання стандартів обміну даними, а також можливість розширення функціоналу без переробки основної архітектури.

2.5.7 Масштабованість, тестування та розвиток

Масштабованість API досягається завдяки модульній структурі, стандартизації підходів, використанню сучасних інструментів тестування (unit, integration, e2e-тести), автоматизованому деплоюменту та можливості

					КР.ІІЗ-4220.ІІЗ	Арк.
						46
Зм.	Арк.	№ докум.	Підпис	Дата		-

горизонтального масштабування серверної частини. Кожен новий модуль або функція інтегруються у систему через чітко визначені контракти (інтерфейси), що виключає виникнення конфліктів і спрощує підтримку коду у великих командах.

2.5.8 Практичні сценарії використання та інструментарій

API є не лише технічним засобом взаємодії, а й потужним інструментом автоматизації бізнес-процесів. Наприклад, автор може подати рукопис, завантажити файли, відстежувати статус розгляду через особистий кабінет, а редактор – призначати рецензентів, отримувати аналітику, контролювати строки виконання. Рецензент, у свою чергу, дистанційно працює з рукописом, залишає коментарі, завантажує відмічені файли, а система автоматично інформує про дедлайни, зміни статусу, необхідність доопрацювання.

					КР.ІІЗ-4220.ІІЗ	Арк.
						47
Зм.	Арк.	№ докум.	Підпис	Дата		-

3 РОЗРОБКА СИСТЕМИ ПІДГОТОВКИ НАУКОВИХ ВИДАНЬ

3.1 Опис структури розробки проєкту

3.1.1 Архітектурні принципи та підхід до структурування коду

Основний принцип організації коду – модульність. Кожен функціональний блок системи реалізований окремим модулем, що містить усі необхідні для роботи компоненти: контролери, сервіси, DTO (Data Transfer Object), сутності (entity) для ORM і, якщо потрібно, власні константи, декоратори або утиліти [].

Проєкт реалізований з таких компонентів:

- Controller (контролери) відповідають за обробку HTTP-запитів, взаємодію з користувачем через REST API, валідацію та делегування бізнес-логіки у відповідні сервіси.
- Service (сервіси) інкапсулюють бізнес-логіку модуля, роботу з базою даних, перевірку ролей, складні сценарії взаємодії з іншими модулями.
- DTO – визначають чіткі типи даних для запитів та відповідей, забезпечують типізацію, валідацію та захист від некоректних даних.
- Entity (сутності) – це моделі для роботи з ORM (TypeORM), які відображаються у відповідні таблиці бази даних.

Кожен модуль максимально ізольований: він містить усі свої залежності й не має прямого доступу до внутрішньої реалізації інших модулів, що забезпечує зручність супроводу та тестування [7].

Нижче наведена фактична структура директорій і файлів проєкту (див. рисунок 3.1).

					КР.ІІЗ-4220.ІІЗ	Арк.
						48
Зм.	Арк.	№ докум.	Підпис	Дата		-

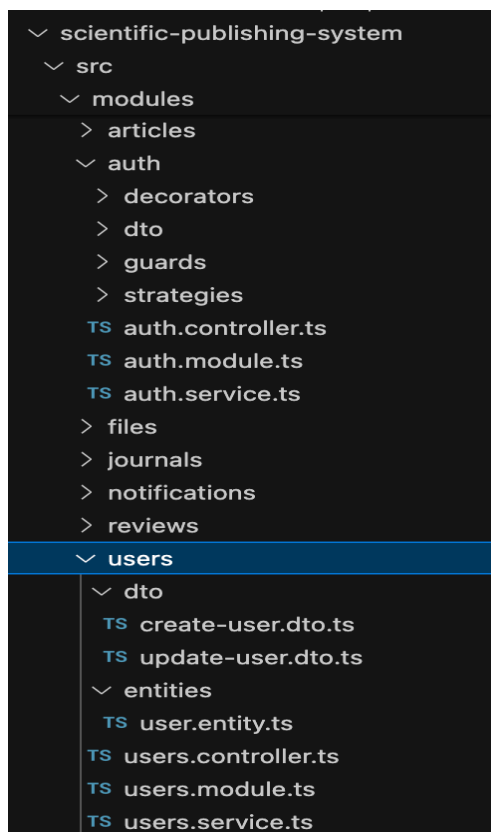


Рисунок 3.1 – Приклад структури модулів проекту

Опис основних директорій та файлів:

- users/ – модуль для управління користувачами (створення, редагування, ролі, доступи).
- auth/ – модуль аутентифікації та авторизації (JWT, захист маршрутів, Guards, декоратори).
- articles/ – робота зі статтями (CRUD, статуси, зв'язки з авторами та рецензентами).
- journals/ – модуль журналів, у яких публікуються статті.
- reviews/ – система призначення і фіксації рецензій, інтеграція з модулями users та articles.
- files/ – завантаження, зберігання та видача файлів (рукописи, рецензії, додатки).

- notifications/ – email-сповіщення про події в системі (призначення рецензента, зміна статусу статті тощо).
- common/ використовується для повторно використовуваних конструкцій: спільних DTO для пагінації, базових класів для помилок, утиліт для формування відповідей тощо.
- config/ містить окремі файли конфігурації (наприклад, database.config.ts, jwt.config.ts), що дозволяє централізовано керувати параметрами підключення в різних середовищах (development, production, test).
- test/ для unit- й e2e-тестів: структура папки повторює структуру основних модулів, що дозволяє легко знаходити тести для кожної частини системи.

3.1.2 Логіка організації коду

Через те, що система є модульною, кожен файл у модулі підключає лише ті сервіси, DTO та сутності, які йому потрібні, і не залежить від внутрішньої реалізації інших модулів. Це дозволяє легко додавати нову функціональність, переносити або видаляти модулі без ризику зламати проєкт. Приклад типової структури одного модуля, розберемо на прикладі модуля users (див. рисунок 3.2).

Модуль users організований за принципами модульності NestJS і містить усі необхідні для роботи компоненти в межах однієї папки. Це забезпечує ізольованість, зручність підтримки та масштабованість.

- Папка dto/ -- у цій папці містяться файли для Data Transfer Objects – спеціальних класів, які описують структуру даних, що передаються у запитах на створення чи оновлення користувача.
- create-user.dto.ts – DTO для створення користувача, визначає, які поля має надавати клієнт для реєстрації або додавання нового користувача.
- update-user.dto.ts – DTO для оновлення користувача, визначає, які поля можна змінювати (наприклад, ім'я, email, роль).

					КР.ПЗ-4220.ПЗ	Арк.
						50
Зм.	Арк.	№ докум.	Підпис	Дата		-

- Папка `entities/` -- тут зберігаються entity-класи, які відповідають таблицям бази даних. Вони визначають структуру збережених у БД об'єктів.
- `user.entity.ts` – Entity, що описує користувача: поля, типи, зв'язки з іншими сутностями (наприклад, ролі, статті тощо).
- `users.controller.ts` – Цей файл містить контролер – клас, який відповідає за обробку HTTP-запитів, пов'язаних із користувачами. Контролер приймає запити від клієнта, викликає потрібні сервіси та повертає результати.

Типові методи контролера `user`:

- створення користувача (POST `/users`)
- отримання списку користувачів (GET `/users`)
- отримання одного користувача (GET `/users/:id`)
- оновлення користувача (PATCH `/users/:id`)
- видалення користувача (DELETE `/users/:id`)

Опис сервісу користувачів:

- `users.service.ts` – сервіс містить усю бізнес-логіку, пов'язану з користувачами. Він взаємодіє з базою даних через entity, реалізує перевірки, обробку помилок, складні сценарії пошуку й зміни даних. Контролер делегує всі складні операції саме в цей клас.
- `users.module.ts` – це файл оголошення модуля NestJS. Тут підключаються всі частини модуля `users` – контролер, сервіс, entity, а також імпортується зовнішні модулі, якщо це потрібно (наприклад, `TypeOrmModule` для роботи з базою даних).

Модуль реєструє свої провайдери та експортує необхідні сервіси для використання в інших модулях. Загалом, така структура дозволяє тримати всю логіку, пов'язану з користувачами, в одному ізольованому місці. Це робить код чистим, зрозумілим і дуже зручним для розширення або рефакторингу.

					КР.ІІЗ-4220.ІІЗ	Арк.
						51
Зм.	Арк.	№ докум.	Підпис	Дата		-

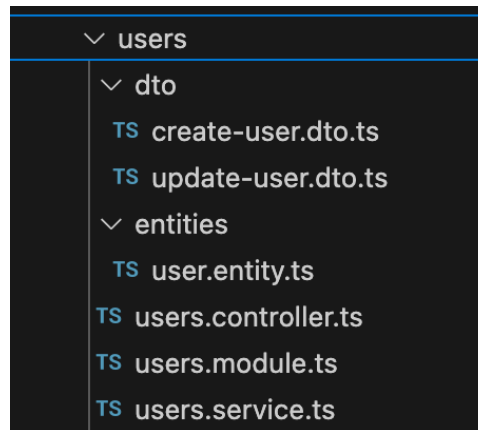


Рисунок 3.2 – Приклад структури модуля users

3.2 Реалізація основних модулів

Система побудована за модульною архітектурою. Кожен функціональний блок представлено окремим модулем, який містить усі необхідні для роботи компоненти: контролери, сервіси, DTO, entity та допоміжні файли. Модулі ізольовані, взаємодіють між собою лише через відкриті сервіси та інтерфейси, що забезпечує масштабованість і спрощує тестування.

Users (Користувачі) – модуль відповідає за створення, оновлення, перегляд і видалення користувачів, а також за керування ролями та профілями.

Приклад контролера:

```

@Controller('users')
export class UsersController {
  constructor(private readonly userService: UsersService) {}

  @Post()
  create(@Body() createUserDto: CreateUserDto) {
    return this.userService.create(createUserDto);
  }

  @Get()
  findAll() {
    return this.userService.findAll();
  }

  @Get('/:id')
  findOne(@Param('id') id: string) {
    return this.userService.findOne(id);
  }
}

```

```

    }

    @Patch('/:id')
    update(@Param('id') id: string, @Body() updateUserDto:
UpdateUserDto) {
        return this.usersService.update(id, updateUserDto);
    }

    @Delete('/:id')
    remove(@Param('id') id: string) {
        return this.usersService.remove(id);
    }
}

```

Використовується в auth для пошуку та перевірки користувачів, у reviews для призначення рецензентів.

Auth (Аутентифікація та авторизація) – забезпечує вхід користувачів у систему, видачу JWT-токенів, контроль доступу до маршрутів, рольову авторизацію.

Приклад використання DI:

```

// auth.service.ts
constructor(
    private usersService: UsersService,
    private jwtService: JwtService,
) {}

```

AuthService використовує UsersService для пошуку та перевірки користувача. Guards і декоратори використовуються в контролерах інших модулів для захисту маршрутів.

Articles (Статті) – управління науковими статтями – створення, редагування, перегляд, зміна статусів, зв'язок із журналами та користувачами.

```

// articles.controller.ts
@Controller('articles')
export class ArticlesController {
    constructor(private readonly articlesService: ArticlesService) {}

    @Post()
    @UseGuards(JwtAuthGuard)
    create(@Body() dto: CreateArticleDto) {
        return this.articlesService.create(dto);
    }

    @Get('/:id')

```

					КР.ІІЗ-4220.ІІЗ	Арк.
						53
Зм.	Арк.	№ докум.	Підпис	Дата		-

```

        findOne(@Param('id') id: string) {
            return this.articlesService.findOne(id);
        }
    }
}

```

Використовується в reviews для призначення рецензентів на статтю, у files для прив'язки файлів.

Journals (Журнали) – модуль відповідає за створення, редагування, перегляд і видалення наукових журналів, у яких публікуються статті. Журнали об'єднують статті за тематикою, мають власні атрибути (назва, опис, ISSN тощо), а також можуть бути пов'язані з редакторами.

Структура контролера journals:

```

// journals.controller.ts
@Controller('journals')
export class JournalsController {
    constructor(private readonly journalsService: JournalsService) {}

    @Post()
    @UseGuards(JwtAuthGuard, RolesGuard)
    @Roles('admin')
    create(@Body() createJournalDto: CreateJournalDto) {
        return this.journalsService.create(createJournalDto);
    }

    @Get()
    findAll() {
        return this.journalsService.findAll();
    }

    @Get('/:id')
    findOne(@Param('id') id: string) {
        return this.journalsService.findOne(id);
    }

    @Patch('/:id')
    @UseGuards(JwtAuthGuard, RolesGuard)
    @Roles('admin')
    update(@Param('id') id: string, @Body() updateJournalDto:
UpdateJournalDto) {
        return this.journalsService.update(id, updateJournalDto);
    }

    @Delete('/:id')
    @UseGuards(JwtAuthGuard, RolesGuard)
    @Roles('admin')
    remove(@Param('id') id: string) {
        return this.journalsService.remove(id);
    }
}

```

```
}
```

Модуль журналів використовується в `articles` для прив'язки статті до певного журналу через `journalId`. Всі операції з журналами дозволені лише адміністраторам, що реалізується через `guards` і декоратори ролей.

`Reviews` (Рецензії) – модуль організовує процес рецензування статей. Він дозволяє призначати рецензентів, фіксувати статус рецензії, зберігати коментарі, а також приймати або відхиляти статті за підсумками рецензування. В архітектурі модуля передбачено гнучку рольову модель: призначати рецензентів можуть лише адміністратори і редактори, самі рецензенти можуть лише змінювати статус і додавати відгуки.

Приклад контролера:

```
// reviews.controller.ts
@Controller('reviews')
export class ReviewsController {
  constructor(private readonly reviewsService: ReviewsService) {}

  @Post()
  @UseGuards(JwtAuthGuard, RolesGuard)
  @Roles('admin', 'editor')
  create(@Body() createReviewDto: CreateReviewDto) {
    return this.reviewsService.create(createReviewDto);
  }

  @Get()
  @UseGuards(JwtAuthGuard)
  findAll(@Query('articleId') articleId?: string,
    @Query('reviewerId') reviewerId?: string) {
    return this.reviewsService.findAll({ articleId, reviewerId });
  }

  @Get('/:id')
  @UseGuards(JwtAuthGuard)
  findOne(@Param('id') id: string) {
    return this.reviewsService.findOne(id);
  }

  @Patch('/:id')
  @UseGuards(JwtAuthGuard, RolesGuard)
  @Roles('admin', 'editor', 'reviewer')
  update(@Param('id') id: string, @Body() updateReviewDto:
    UpdateReviewDto) {
    return this.reviewsService.update(id, updateReviewDto);
  }
}
```

					КР.ІІЗ-4220.ІІЗ	Арк.
						55
Зм.	Арк.	№ докум.	Підпис	Дата		-

ReviewsService використовує ArticlesService для перевірки існування статті, UsersService для перевірки рецензента, а також викликає NotificationsService для надсилання повідомлень про нові призначення [8].

Files (Файли) – модуль забезпечує завантаження, зберігання та видачу супровідних файлів, які прикріплюються до статей або рецензій (рукописи, ілюстрації, рецензійні документи). Тут реалізовано валідацію типу та розміру файлів, а також можливість вибору способу зберігання (файлова система, cloud storage).

Приклад сервісу:

```
// files.service.ts
@Injectable()
export class FilesService {
  async uploadFile(file: Express.Multer.File): Promise<FileEntity>
  {
    // Зберігаємо файл у файловій системі/хмарі
    // Повертаємо метадані файлу для збереження у базі
  }

  async getFile(id: string): Promise<FileEntity> {
    // Пошук файлу по id та видача його/шляху для завантаження
  }
}
```

FilesService взаємодіє з ArticlesService і ReviewsService для перевірки прав на додавання/видалення файлів, а також для прив'язки файлів до відповідних сутностей.

Всі модулі побудовані із застосуванням DI (Dependency Injection) NestJS: залежності між модулями оголошуються в конструкторах, сервіси імпортуються через модульні імпорти, що забезпечує максимальну гнучкість і можливість підміни компонентів при тестуванні. Наприклад, ReviewsService у своєму конструкторі отримує посилання на ArticlesService, UsersService.

```
// reviews.service.ts
constructor(
  @InjectRepository(Review)
  private reviewsRepository: Repository<Review>,
  private articlesService: ArticlesService,
  private usersService: UsersService,
  private notificationsService: NotificationsService,
) {}
```

					КР.ІІЗ-4220.ІІЗ	Арк.
						56
Зм.	Арк.	№ докум.	Підпис	Дата		-

3.3 Інтеграція системи авторизації та аутентифікації

3.3.1 Конфігурація JwtModule та стратегія

JWT-токени створюються та перевіряються за допомогою JwtModule, який конфігурується в модулі авторизації:

```
// auth.module.ts
import { JwtModule } from '@nestjs/jwt';

@Module({
  imports: [
    JwtModule.register({
      secret: process.env.JWT_SECRET,
      signOptions: { expiresIn: '1d' },
    }),
    UsersModule,
  ],
  controllers: [AuthController],
  providers: [AuthService, JwtStrategy, RolesGuard],
  exports: [AuthService],
})
export class AuthModule {}
```

JwtStrategy визначає, як із токена витягуються дані користувача для подальшої перевірки у guards:

```
// jwt.strategy.ts
@Injectable()
export class JwtStrategy extends PassportStrategy(Strategy) {
  constructor() {
    super({
      jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken(),
      ignoreExpiration: false,
      secretOrKey: process.env.JWT_SECRET,
    });
  }

  async validate(payload: any) {
    return { userId: payload.sub, email: payload.email, roles:
payload.roles };
  }
}
```

					КР.ІІЗ-4220.ІІЗ	Арк.
						57
Зм.	Арк.	№ докум.	Підпис	Дата		-

3.3.2 Захист маршрутів через guards та рольова авторизація

Для обмеження доступу до захищених маршрутів використовується guard `JwtAuthGuard`, який наслідує вбудований `AuthGuard('jwt')`:

```
// jwt-auth.guard.ts
@Injectable()
export class JwtAuthGuard extends AuthGuard('jwt') {}
```

Використання guard у контролерах виглядає так:

```
// articles.controller.ts
@UseGuards(JwtAuthGuard)
@Post()
create(@Body() createArticleDto: CreateArticleDto) {
  return this.articlesService.create(createArticleDto);
}
```

Тобто доступ до створення статті мають лише автентифіковані користувачі, які надали валідний JWT-токен у заголовку `Authorization` [7].

Для подальшого розмежування прав доступу між ролями (наприклад, щоб лише адміністратор міг видаляти журнали або призначати рецензентів) використовується кастомний декоратор `@Roles` і guard `RolesGuard`.

Декоратор дозволяє явно зазначити, які ролі мають право викликати певний метод контролера:

```
// journals.controller.ts
@UseGuards(JwtAuthGuard, RolesGuard)
@Roles('admin')
@Delete(':id')
remove(@Param('id') id: string) {
  return this.journalsService.remove(id);
}
```

Реалізація декоратора:

```
// roles.decorator.ts
import { SetMetadata } from '@nestjs/common';
export const Roles = (...roles: string[]) => SetMetadata('roles', roles);
```

Guard для перевірки ролі виглядає так:

```
// roles.guard.ts
@Injectable()
export class RolesGuard implements CanActivate {
  constructor(private reflector: Reflector) {}
```

```

    canActivate(context: ExecutionContext): boolean {
        const requiredRoles =
            this.reflector.getAllAndOverride<string[]>('roles', [
                context.getHandler(),
                context.getClass(),
            ]);
        if (!requiredRoles) {
            return true;
        }
        const { user } = context.switchToHttp().getRequest();
        return requiredRoles.some((role) =>
            user.roles?.includes(role));
    }
}

```

Таким чином, при кожному запиті guard перевіряє, чи роль користувача з JWT-токена входить у перелік дозволених для цього ендпоінта [6, 11].

3.3.3 Взаємодія AuthService з UsersService

AuthService використовує UsersService для пошуку користувача за email, перевірки пароля, отримання ролей та формування даних для токена. Це забезпечує чітке розділення відповідальності: вся логіка роботи з користувачами зосереджена у users, а логіка аутентифікації – в auth (див додаток В).

```

// auth.service.ts (фрагмент)
constructor(
    private usersService: UsersService,
    private jwtService: JwtService,
) {}

```

3.3.4 Процес логіну, оновлення токена, перевірки доступу

Користувач надсилає email і пароль на /auth/login. – AuthService перевіряє користувача через UsersService, порівнює паролі. У разі успіху генерується JWT-токен з id, email, ролями. Усі подальші захищені запити містять цей токен у заголовку Authorization. JwtAuthGuard автоматично перевіряє валідність токена і підвантажує user у запит. RolesGuard додатково перевіряє, чи має роль

					КР.ІІЗ-4220.ІІЗ	Арк.
						59
Зм.	Арк.	№ докум.	Підпис	Дата		-

користувача доступ до конкретного метода. При спробі доступу без токена, з невалідним токеном або недостатньою роллю повертається помилка 401/403.

Розглянемо використання контролера для створення журналу тільки для адміністратора:

```
@Post()
@UseGuards(JwtAuthGuard, RolesGuard)
@Roles('admin')
create(@Body() createJournalDto: CreateJournalDto) {
    return this.journalsService.create(createJournalDto);
}
```

Зміна статусу рецензії (тільки рецензент або редактор):

```
@Patch('/:id')
@UseGuards(JwtAuthGuard, RolesGuard)
@Roles('admin', 'editor', 'reviewer')
update(@Param('id') id: string, @Body() updateReviewDto:
UpdateReviewDto) {
    return this.reviewsService.update(id, updateReviewDto);
}
```

Отримання списку статей – будь-який автентифікований користувач:

```
@Get()
@UseGuards(JwtAuthGuard)
findAll() {
    return this.articlesService.findAll();
}
```

Уся система побудована так, щоб навіть при додаванні нових типів користувачів або сценаріїв доступу достатньо було лише додати нову роль і вказати її в декораторі, не змінюючи основної логіки guards чи контролерів.

3.4 Приклади сценаріїв та REST API

Створення користувача (реєстрація) – коли новий користувач хоче зареєструватися, він надсилає POST-запит на /api/auth/register, в якому містить своє ім'я, прізвище, email, пароль та інші дані. Контролер приймає ці дані, створює DTO-об'єкт і передає його в сервіс авторизації. Сервіс перевіряє, чи не існує вже користувача з такою поштою, хешує пароль, формує нову entity користувача та зберігає її у базі. Якщо все вдалося – користувач отримує

					КР.ІІЗ-4220.ІІЗ	Арк.
						60
Зм.	Арк.	№ докум.	Підпис	Дата		-

повідомлення про успішну реєстрацію. Далі, якщо система підтримує підтвердження email, на пошту надсилається лист із посиланням для верифікації [7].

- Контролер – приймає й валідує дані з тіла запиту.
- DTO – гарантує, що усі поля коректні.
- Сервіс – реалізує перевірки, хешування, збереження, генерацію токена для підтвердження email.
- Entity – описує, як саме користувач збережеться у БД.

Вхід у систему (логін) - щоб увійти, користувач надсилає POST-запит на `/api/auth/login` із email та паролем. Контролер формує DTO і передає сервісу, який знаходить користувача за email, перевіряє пароль і, якщо все збігається, генерує JWT-токен. Користувач отримує цей токен у відповіді – надалі всі захищені запити він надсилатиме з цим токеном у заголовку `Authorization`. JWT – несе в собі ID, email та ролі користувача, і в подальшому токен використовується як “ключ” до захищених ресурсів (наприклад, перегляд профілю, створення статті).

Всі запити до важливих ресурсів (наприклад, отримання списку користувачів через `/api/users`, створення статті через `/api/articles`, керування журналами через `/api/journals`) перевіряються guard-ами. Вони аналізують JWT-токен із заголовка. Якщо токен коректний і в ньому роль дозволяє виконати дію – запит проходить далі.

Guards – працюють на етапі до виклику контролера, “відсікають” неавторизованих або тих, хто не має права.

Ролі можна обмежити через декоратори – наприклад, тільки адміністратор може видалити журнал.

Коли потрібно змінити дані користувача (PUT `/api/users/{id}`) чи видалити його (DELETE `/api/users/{id}`), система знову перевіряє токен і роль користувача (наприклад, користувач може змінювати лише себе, а адміністратор – будь-кого).

					КР.ІІЗ-4220.ІІЗ	Арк.
						61
Зм.	Арк.	№ докум.	Підпис	Дата		-

Контролер отримує ID користувача з шляху запиту, DTO з тіла, і передає сервісу, який реалізує перевірки, оновлює або видаляє entity у базі.

Щоб створити статтю, потрібен POST-запит на `/api/articles` з усіма даними (назва, анотація, журнал, автори). Система автоматично підставляє ID користувача з JWT-токена як `submitterId`. Сервіс перевіряє, чи існує журнал, чи користувач має право публікувати, створює нову entity статті з потрібним статусом (“submitted”, “draft” тощо) [9].

Переглядати статті можна GET-запитами на `/api/articles`, а деталізовано – через `/api/articles/{id}`. Фільтрація за журналом, статусом, пошук за назвою чи анотацією – все це реалізовано через query-параметри.

Оновлення чи видалення статті (PATCH/DELETE) також обмежується правами – автор може змінювати лише свою статтю, редактор – всі, адміністратор – також всі.

Файли (рукописи, супровідні матеріали) до статей завантажуються через POST `/api/articles/upload` чи аналогічний endpoint для загальних файлів. Користувач надсилає файл як `multipart/form-data`, контролер обробляє файл, перевіряє права, сервіс зберігає файл на диск або у хмару, створює entry в базі.

Отримати список файлів для статті можна через GET `/api/articles/{articleId}/files`. Видалення чи оновлення файлів також захищене – доступ лише автору або адміністратору.

Керування журналами (створення, оновлення, видалення) та їхніми номерами реалізовано через `/api/journals` і відповідні endpoints для issues. Тут все аналогічно: контролер приймає DTO, сервіс перевіряє права, entity зберігається у базі. Додавати номери журналу, редагувати їх, публікувати новий випуск – всі ці дії доступні лише редакторам і адміністраторам.

Адміністратор або редактор призначає рецензента для статті через POST `/api/reviews`, вказуючи ID статті, рецензента, термін виконання. Сервіс перевіряє, чи є стаття, рецензент, чи не призначено вже цю статтю цьому рецензенту.

					КР.ІІЗ-4220.ІІЗ	Арк.
						62
Зм.	Арк.	№ докум.	Підпис	Дата		-

Рецензент отримує завдання, може змінити статус (PATCH), залишити коментарі. Всі ці запити обробляються через контролер рецензій, який також перевіряє права [18].

Коли відбувається важлива подія (новий review, зміна статусу статті, публікація issue тощо), інший сервіс створює сповіщення через POST /api/notifications.

Користувач може отримати всі свої сповіщення через GET /api/notifications?userId=..., відмітити як прочитані (PATCH), видалити непотрібні (DELETE).

Сценарій повного життєвого циклу:

- Реєстрація: Користувач реєструється.
- Логін: Користувач входить, отримує JWT.
- Подача статті: Автор створює статтю, додає файли.
- Призначення рецензента: Редактор призначає рецензента, той отримує сповіщення.
- Рецензування: Рецензент залишає відгук, змінює статус рецензії.
- Редактор приймає рішення: Стаття отримує новий статус (“accepted”, “revisions_needed” тощо).
- Публікація випуску: Адміністратор чи редактор формує новий випуск журналу.
- Усі залучені отримують сповіщення через систему повідомлень про статус роботи

Уся ця логіка, захищеність і послідовність забезпечуються на рівні контролерів (які приймають і маршрутизують запити), DTO (які валідують і описують дані), сервісів (які реалізують бізнес-логіку й перевірки) та guards (які забезпечують безпеку й рольову модель доступу).

					КР.ІІЗ-4220.ІІЗ	Арк.
						63
Зм.	Арк.	№ докум.	Підпис	Дата		-

3.5 Створення інтерфейсу системи

3.5.1 Технологічний стек фронтенд-частини

Для створення клієнтської частини системи використано наступні технології:

- React.js – бібліотека для створення інтерактивних інтерфейсів користувача, що дозволяє розробляти компонентний і декларативний код.
- Material UI – бібліотека компонентів, що реалізує принципи Material Design та забезпечує сучасний вигляд інтерфейсу.
- React Router – бібліотека для навігації між сторінками односторінкового додатку, що забезпечує навігацію без перезавантаження сторінки.
- Formik та Yup – бібліотеки для створення та валідації форм, що спрощують роботу з формами введення даних.
- Axios – бібліотека для HTTP-запитів до серверної частини, що полегшує взаємодію з API.
- JWT Decode – бібліотека для роботи з JSON Web Token, що використовується для автентифікації користувачів.
- React Quill – редактор форматованого тексту для написання анотацій та інших текстових матеріалів.
- React Toastify – компонент для відображення сповіщень користувачу про результати операцій.

3.5.2 Архітектура фронтенд-частини

Архітектура клієнтської частини системи базується на компонентному підході з використанням контексту для управління станом додатку [10]. Основні архітектурні рішення:

					КР.ІІЗ-4220.ІІЗ	Арк.
						64
Зм.	Арк.	№ докум.	Підпис	Дата		-

- Компонентна структура – весь інтерфейс розділено на незалежні компоненти, що полегшує підтримку та розширення функціональності.
- Контекст автентифікації – реалізовано через React Context API, що дозволяє зберігати інформацію про поточного користувача та його права доступу в глобальному стані додатку.
- Сервісний шар – абстрагує взаємодію з API, інкапсулюючи логіку HTTP-запитів та обробки відповідей.
- Захищені маршрути – реалізовано компонент ProtectedRoute, який забезпечує доступ до сторінок відповідно до ролей користувача.
- Реактивність інтерфейсу – стан компонентів автоматично оновлюється при зміні даних, забезпечуючи послідовність відображення інформації.

3.5.3 Основні компоненти системи

У фронтенд-частині системи наукових публікацій можна виділити декілька основних компонентів, які забезпечують її функціонування. Кожен компонент відповідає за певний аспект роботи системи.

Компоненти автентифікації забезпечують роботу з обліковими записами користувачів. Вони включають форми реєстрації з можливістю вибору ролі (автор, рецензент, редактор, читач), форму входу в систему, сторінки для відновлення паролю та підтвердження електронної пошти. Також реалізовано механізми захисту маршрутів, щоб неавторизовані користувачі не мали доступу до захищених сторінок.

Компоненти для роботи з журналами дозволяють переглядати список доступних наукових видань, отримувати детальну інформацію про кожен журнал (включаючи вимоги до публікацій, політику рецензування, контактну інформацію). Редактори мають доступ до панелі адміністрування журналів, де можуть керувати випусками, налаштовувати параметри та визначати політики публікацій.

					КР.ІІЗ-4220.ІІЗ	Арк.
						65
Зм.	Арк.	№ докум.	Підпис	Дата		-

Для роботи зі статтями розроблено низку компонентів, які дозволяють авторам подавати рукописи через покроковий процес (вибір журналу, заповнення метаданих, додавання співавторів, завантаження файлів). Автори можуть переглядати стан своїх статей, відповідати на зауваження рецензентів та вносити виправлення. Редактори використовують ці компоненти для опрацювання статей та керування процесом публікації.

Компоненти рецензування забезпечують весь цикл експертної оцінки статей. Редактори призначають рецензентів, а ті заповнюють форми з оцінками за різними критеріями та пишуть коментарі. Система автоматично збирає та узагальнює результати рецензування, допомагаючи редакторам приймати обґрунтовані рішення щодо публікації.

Адміністративні компоненти надають інструменти для управління користувачами (створення, редагування, блокування), налаштування параметрів системи та моніторингу активності. Адміністратори можуть переглядати журнали дій користувачів, вирішувати проблемні ситуації та налаштовувати глобальні параметри платформи.

До допоміжних компонентів належать:

- Система сповіщень, яка інформує користувачів про важливі події та дії, що потребують уваги
- Компоненти завантаження та перегляду файлів різних форматів (PDF, DOCX тощо)
- Пошукові інтерфейси з можливістю фільтрації за різними критеріями
- Панелі навігації та інформаційні панелі для різних типів користувачів

3.5.4 Взаємодія з API

Для забезпечення взаємодії фронтенд-частини з серверною реалізовано сервісний шар, який включає:

					КР.ІІЗ-4220.ІІЗ	Арк.
						66
Зм.	Арк.	№ докум.	Підпис	Дата		-

Базовий сервіс API – налаштований клієнт Axios з перехоплювачами для автоматичного додавання токенів автентифікації та обробки помилок.

Спеціалізовані сервіси:

- AuthService – методи для автентифікації та управління профілем
- JournalService – взаємодія з API журналів та випусків
- ArticleService – операції зі статтями
- ReviewService – управління рецензіями
- FileService – завантаження та управління файлами
- NotificationService – робота зі сповіщеннями
- UserService – управління користувачами

Обробка помилок – централізована система відображення помилок з інформативними повідомленнями для користувачів.

3.5.5 Особливості користувацького інтерфейсу

Інтерфейс користувача розроблено з урахуванням принципів зручності використання (UX) та гарного дизайну (UI) (див. рисунок 3.3):

- Інтуїтивна навігація – реалізовано зрозумілу структуру меню, хлібні крихти та допоміжні елементи навігації.
- Покроковий процес подання статті – розділено складний процес подання статті на логічні кроки з проміжним збереженням даних.
- Диференційований доступ – інтерфейс автоматично адаптується до ролі користувача, показуючи тільки релевантні функції та елементи..
- Доступність – враховано вимоги доступності для користувачів з різними потребами [24].

					КР.ІІЗ-4220.ІІЗ	Арк.
						67
Зм.	Арк.	№ докум.	Підпис	Дата		-

The image shows a login interface with the following elements:

- Title:** Login
- Subtitle:** Sign in to your Scientific Publishing System account
- Form Fields:**
 - Email (text input)
 - Password (password input)
- Buttons/Links:**
 - [Forgot password?](#) (link)
 - SIGN IN** (button)
 - OR (text separator)
 - [Don't have an account? Register](#) (link)

Рисунок 3.3 – сторінка логіну користувача

3.5.6 Забезпечення безпеки

В клієнтській частині реалізовано наступні механізми безпеки:

- Токенна автентифікація – використання JSON Web Tokens (JWT) для безпечної автентифікації та авторизації.
- Захищені маршрути – перевірка прав доступу перед відображенням захищеного контенту.
- Валідація даних – перевірка всіх вхідних даних на клієнтському рівні перед відправкою на сервер.
- Безпечне зберігання даних – використання localStorage для зберігання токенів з визначеним терміном дії.
- Захист від CSRF та XSS атак – реалізовано базові механізми захисту через налаштування HTTP-заголовків та санітизацію даних.

3.5.7 Результати розробки інтерфейсу

В результаті розробки створено повноцінний, інтуїтивно зрозумілий інтерфейс користувача (див рисунок 3.3), який дозволяє:

					КР.ІІЗ-4220.ІІЗ	Арк.
						68
Зм.	Арк.	№ докум.	Підпис	Дата		-

- Авторам: подавати статті, відстежувати їх статус та взаємодіяти з редакцією
- Рецензентам: переглядати призначені статті та створювати детальні рецензії
- Редакторам: керувати процесом публікації, призначати рецензентів та ухвалювати рішення
- Адміністраторам: налаштовувати систему та керувати всіма аспектами її роботи

Всі компоненти інтерфейсу відповідають сучасним вимогам веб-розробки, забезпечуючи швидку, безпечну та зручну роботу користувачів з системою наукових публікацій.

3.6 Контейнеризація системи

3.6.1 Вступ до контейнеризації

Контейнеризація – це метод віртуалізації на рівні операційної системи, який дозволяє запускати ізольовані процеси в спільному середовищі. Docker, як платформа для розробки, доставки та запуску додатків у контейнерах, забезпечує стандартизований підхід до упаковки програмного забезпечення.

Основними перевагами контейнеризації є:

- Ізоляція залежностей – кожен компонент системи працює у власному контейнері з усіма необхідними бібліотеками
- Однорідність середовища – усунення проблем "на моєму комп'ютері працює"
- Швидке розгортання – контейнери запускаються за секунди
- Ефективне використання ресурсів – контейнери споживають менше ресурсів, ніж віртуальні машини

					КР.ІІЗ-4220.ІІЗ	Арк.
						69
Зм.	Арк.	№ докум.	Підпис	Дата		-

- Масштабованість – можливість горизонтального масштабування системи

3.6.2 Multi-stage building

Для оптимізації процесу створення контейнерів використовується підхід multi-stage building (багатоетапна збірка), який дозволяє:

- Зменшити розмір підсумкового образу
- Покращити безпеку, не включаючи інструменти розробки у виробничий контейнер
- Оптимізувати процес кешування під час збірки
- Розділити етапи компіляції та виконання
- Цей підхід особливо важливий для нашої системи, яка складається з фронтенд-частини на React та бекенд-частини на Node.js/NestJS.

3.6.3 Контейнеризація бекенд-частини

Для бекенду системи наукових публікацій створено наступний Dockerfile з використанням multi-stage building:

```
# Stage 1: Build stage
FROM node:18-alpine AS builder

# Set working directory
WORKDIR /app

# Copy package files and install dependencies
COPY package*.json ./
RUN npm ci

# Copy source code
COPY . .

# Build the application
RUN npm run build

# Stage 2: Production stage
FROM node:18-alpine AS production
```

					КР.ІІЗ-4220.ІІЗ	Арк.
						70
Зм.	Арк.	№ докум.	Підпис	Дата		-

```
# Set working directory
WORKDIR /app

# Set environment variable
ENV NODE_ENV=production

# Copy only necessary files from the builder stage
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/package*.json ./

# Expose API port
EXPOSE 3000

# Set non-root user
USER node

# Start the application
CMD ["node", "dist/main"]
```

Цей Dockerfile складається з двох етапів:

- Етап збірки (builder): На цьому етапі встановлюються всі залежності, включаючи devDependencies, та компілюється TypeScript-код.
- Виробничий етап (production): У фінальний образ копіюються лише необхідні файли – скомпільований код, виробничі залежності та файли конфігурації.

Такий підхід дозволяє значно зменшити розмір підсумкового образу (з ~1 ГБ до ~200 МБ) та підвищити безпеку, оскільки у виробничому середовищі не зберігається вихідний код і засоби розробки.

3.6.4 Контейнеризація фронтенд-частини

Для фронтенд-частини застосовано подібний підхід, але з використанням NGINX для обслуговування статичного контенту:

```
# Stage 1: Build stage
FROM node:18-alpine AS builder

# Set working directory
WORKDIR /app

# Copy package files and install dependencies
```

					КР.ІІЗ-4220.ІІЗ	Арк.
						71
Зм.	Арк.	№ докум.	Підпис	Дата		-

```

COPY package*.json ./
RUN npm ci

# Copy source code
COPY . .

# Build the application
RUN npm run build

# Stage 2: Production stage
FROM nginx:alpine AS production

# Copy built files from the builder stage
COPY --from=builder /app/build /usr/share/nginx/html

# Copy custom NGINX config if needed
COPY nginx.conf /etc/nginx/conf.d/default.conf

# Expose port
EXPOSE 80

# Start NGINX server
CMD ["nginx", "-g", "daemon off;"]

```

Цей Dockerfile також має два етапи:

- Етап збірки (builder): Встановлюються залежності та компілюється React-додаток до оптимізованих статичних файлів.
- Виробничий етап (production): Використовується легкий образ NGINX, куди копіюються лише зібрані статичні файли та конфігурація веб-сервера.

3.6.5 Оркестрація контейнерів з Docker Compose

Для управління взаємодією між контейнерами використовується Docker Compose, що дозволяє оркеструвати різні сервіси системи:

```

version: '3.8'

services:
  # База даних PostgreSQL
  postgres:
    image: postgres:15
    container_name: scientific-publishing-postgres
    restart: always
    volumes:
      - postgres_data:/var/lib/postgresql/data

```

					КР.ІІЗ-4220.ІІЗ	Арк.
						72
Зм.	Арк.	№ докум.	Підпис	Дата		-


```

env_file:
  - .env
environment:
  - POSTGRES_USER=${DB_USER}
  - POSTGRES_PASSWORD=${DB_PASSWORD}
  - POSTGRES_DB=${DB_NAME}
ports:
  - "5432:5432"
networks:
  - scientific-publishing-network
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U ${DB_USER} -d ${DB_NAME}"]
  interval: 10s
  timeout: 5s
  retries: 5

# Бекенд API
backend:
  build:
    context: ./backend
    dockerfile: Dockerfile
    target: production
  container_name: scientific-publishing-backend
  restart: always
  env_file:
    - .env
  environment:
    - NODE_ENV=production
    -
DATABASE_URL=postgresql://${DB_USER}:${DB_PASSWORD}@postgres:5432/${DB_NAME}
  depends_on:
    postgres:
      condition: service_healthy
  networks:
    - scientific-publishing-network
  ports:
    - "3000:3000"

# Фронтенд веб-інтерфейс
frontend:
  build:
    context: ./frontend
    dockerfile: Dockerfile
    target: production
  container_name: scientific-publishing-frontend
  restart: always
  depends_on:
    - backend
  networks:
    - scientific-publishing-network
  ports:
    - "80:80"

# Визначення мереж та томів
networks:

```

					КР.ІІЗ-4220.ІІЗ	Арк.
						73
Зм.	Арк.	№ докум.	Підпис	Дата		-

```
scientific-publishing-network:
  driver: bridge
volumes:
  postgres_data:
    driver: local
```

					КР.ІІЗ-4220.ІІЗ	Арк.
						74
Зм.	Арк.	№ докум.	Підпис	Дата		-

4 БІЗНЕС-ПЛАН ТА ВИТРАТИ НА СИСТЕМУ

4.1 Аналіз ринку та конкурентів

Ринок систем для автоматизації наукового публікування демонструє стійке зростання і перебуває у фазі активної цифрової трансформації. Потреба в таких платформах зумовлена як глобальними тенденціями відкритої науки (Open Science), так і локальними ініціативами університетів, академічних інститутів, наукових видавництв. Сучасний науковий світ стикається з необхідністю забезпечити прозорість, ефективність, швидкість, а також контрольованість усіх етапів життєвого циклу наукової статті: від подачі рукопису, рецензування, редакційної обробки – до публікації, архівації та індексації у світових базах даних.

Обсяг ринку наукового публікування оцінюється у мільярди доларів і щороку зростає, особливо за рахунок розвитку open-access журналів, діджиталізації процесів у науці й освіті, а також збільшення кількості дослідників і наукових проєктів. Відбувається поступова відмова від паперових і “ручних” процедур на користь електронних платформ, які забезпечують автоматизацію, інтеграцію з інституційними репозитаріями, DOI-присвоєння, ORCID-ідентифікацію, боротьбу з плагіатом, аналітику та звітність.

Глобальні корпоративні рішення – це закриті, дорогі, складні у впровадженні системи, орієнтовані на великі комерційні видавництва (наприклад, Elsevier Editorial Manager, ScholarOne Manuscripts, Aries Systems Editorial Manager, eJournalPress) [3]. Їхні переваги – потужна функціональність, інтеграція з екосистемою видавництва, глибока аналітика. Недоліки – висока ціна, складність кастомізації, слабка підтримка локальних мов і специфіки, відсутність відкритих API.

Open-source та університетські платформи – найпопулярнішим представником є Open Journal Systems (OJS). Це безкоштовна система з

					КР.ІІЗ-4220.ІІЗ	Арк.
						75
Зм.	Арк.	№ докум.	Підпис	Дата		-

відкритим кодом, яку використовують тисячі журналів у світі. Її переваги – доступність, активна спільнота, можливість самостійного розгортання. Водночас вона має складний, застарілий інтерфейс, важку процедуру оновлення та обмежені можливості інтеграції з сучасними сервісами (S3, антиплагіат, платіжні шлюзи тощо). Інші представники – Ambra, Janeway, Scholastica.

Молоді SaaS-рішення та нішеві сервіси – це Manuscript Manager, Scholastica, PeerTrack, Morressier, які позиціонують себе як “легкі”, швидкі у впровадженні, зручні для невеликих журналів.

Проблеми, які залишаються невирішеними на ринку:

- Вартість володіння: великі системи недоступні для малих і середніх наукових журналів через високу ціну підписки чи ліцензії.
- Відсутність кастомізації: неможливо адаптувати робочі процеси, інтерфейс чи логіку під потреби конкретного видання.
- Складність інтеграції: більшість сучасних рішень не мають повноцінного API або можливості інтегруватися з локальними сервісами (наприклад, університетськими бібліотеками, національними репозитаріями, платіжними шлюзами).
- Недостатній рівень автоматизації: частина процесів досі залишається “ручними” (розсилки, рецензування, оплата APC, генерація метаданих для індексації).
- Відсутність підтримки української мови, локальних стандартів, вимог до звітності.

Розроблена система вирішує ці проблеми за допомогою сучасної архітектури, яка базується на принципах API-first, microservices-ready, гнучкої авторизації, підтримки різних ролей (автор, редактор, рецензент, адміністратор), інтеграції з S3, email, антиплагіат-сервісами, платіжними системами. Також вона проєктується з урахуванням локальних потреб – підтримка української мови, кастомізація робочих процесів, зручний інтерфейс, можливість швидкого

					КР.ІІЗ-4220.ІІЗ	Арк.
						76
Зм.	Арк.	№ докум.	Підпис	Дата		-

розгортання як у хмарі (AWS, Azure, Google Cloud), так і на власних серверах організацій [1, 21].

Конкурентні переваги перед аналогами:

- Сучасний web-інтерфейс, мобільна адаптивність
- Повноцінний REST API для інтеграцій з іншими системами та автоматизації
- Розширювана модульна архітектура (можна додавати нові модулі/ролі/процеси)
- Легке розгортання в Docker/Kubernetes, підтримка CI/CD
- Підтримка локалізації, кастомізації під вимоги журналу
- Доступна ціна (SaaS або ліцензія), індивідуальна підтримка

Тенденції ринку:

- Зростання кількості open-access журналів і університетських видавництв
- Зростання вимог до безпеки, прозорості peer-review та контролю якості
- Перехід до хмарних рішень та автоматизації всіх етапів життєвого циклу статті
- Збільшення попиту на інтеграцію з міжнародними базами, ORCID, Crossref, антиплагіат-сервісами
- Потреба в гнучких інструментах аналітики й звітності для редакторів та адміністраторів

Scientific Publishing System вписується у ці тенденції, пропонуючи сучасний, зручний, адаптивний і доступний інструмент для автоматизації наукового публікування, що має потенціал для масштабування як на локальному, так і на міжнародному ринку

					КР.ІІЗ-4220.ІІЗ	Арк.
						77
Зм.	Арк.	№ докум.	Підпис	Дата		-

4.2 Цільова аудиторія, сегментація та позиціонування

Scientific Publishing System орієнтована на широкий спектр учасників науково-видавничого процесу, від малих університетських журналів до міжнародних видавництв, що прагнуть модернізувати та автоматизувати роботу з науковими публікаціями. Визначення й розуміння цільової аудиторії є ключовим фактором для успішного маркетингу, підвищення лояльності користувачів і конкурентоспроможності системи.

Основними користувачами системи є редакційні колективи наукових журналів – як окремі видання, так і великі видавничі дома, що керують портфелем журналів. Для них важливі автоматизація рутинних процесів (прийом рукописів, організація peer review, ведення архіву, робота з індексацією), гнучкість налаштувань, прозорість workflow та зручність для редакторів, рецензентів й авторів.

Університети, академічні інститути, наукові організації – часто мають власні наукові видання або збірники праць. Вони зацікавлені у рішеннях, які дозволяють централізовано адмініструвати декілька журналів, інтегрувати систему з інституційними репозиторіями, забезпечити дотримання академічних стандартів та відкритості.

Платформа має бути максимально дружньою до авторів – інтуїтивний процес подачі рукопису, прозора система відстеження статусу, зручність спілкування з редакцією й рецензентами. Для авторів важлива швидкість, надійність та простота взаємодії з платформою.

Адміністратори та ІТ-відділи – відповідальні за впровадження, налаштування, безпеку та підтримку системи. Для них важливі легкість розгортання (Docker, хмара чи on-premise), можливість інтеграції з іншими інформаційними системами, гнучке управління правами доступу, простота резервного копіювання та оновлення [14, 22].

					КР.ІІЗ-4220.ІІЗ	Арк.
						78
Зм.	Арк.	№ докум.	Підпис	Дата		-

Для ефективного просування Scientific Publishing System доцільно сегментувати потенційних клієнтів за кількома ключовими ознаками (див. таблиця 4.1).

За типом організації:

- Університетські та академічні журнали
- Державні наукові установи
- Приватні видавництва
- Громадські і професійні асоціації, що випускають наукові збірники

За розміром і обсягом публікацій:

- Малі журнали (1-2 випуски на рік)
- Середні журнали (3-6 випусків на рік, до 100 статей)
- Великі журнали та видавничі дома (безлімітна кількість видань, сотні й тисячі статей)

За ступенем цифрової зрілості:

- Журнали, що переходять із “ручних”/email-процесів
- Журнали, що шукають заміну застарілим платформам
- Новостворені видання, що відразу прагнуть сучасного автоматизованого рішення

Scientific Publishing System позиціонується як сучасна, гнучка, технологічно досконала платформа, що дозволяє швидко й безболісно автоматизувати всі ключові процеси наукового публікування. Її унікальність підкреслюється такими перевагами:

- Модульність та кастомізація: можливість адаптувати workflow, ролі, інтерфейс під конкретний журнал чи видавництво.
- Сучасний UX: інтуїтивний та адаптивний інтерфейс для всіх ролей, mobile-first підхід.

					КР.ІІЗ-4220.ІІЗ	Арк.
						79
Зм.	Арк.	№ докум.	Підпис	Дата		-

- Легкість впровадження: підтримка Docker, CI/CD, хмарних і локальних розгортань.
- Гнучка інтеграція: наявність API для підключення до зовнішніх сервісів (Crossref, ORCID, S3, email, антиплагіат, платіжні рішення).

Таким чином, Scientific Publishing System є ідеальним вибором для організацій, які прагнуть отримати сучасний, гнучкий та масштабований інструмент для цифровізації процесу наукового публікування, підвищити ефективність редакційної роботи й відповідати сучасним міжнародним стандартам якості (див. таблиця 4.2).

Нижче наведено таблицю сегментації цільової аудиторії та карту позиціонування.

Таблиця 4.1 – Сегментація цільової аудиторії

Сегмент	Опис сегменту	Ключові потреби	Відповідність системи
Університетські журнали	Журнали при ВНЗ, академічних інститутах	Локалізація, інтеграція з репозитаріями, простота	Повна відповідність, готові інтеграції
Державні наукові установи	Видавництва, збірники НАН, МОН	Надійність, звітність, workflow під стандарти	Гнучкі налаштування, модулі звітності
Приватні видавництва	Комерційні журнали, незалежні редакції	Модульність, кастомізація, підтримка платних функцій	SaaS, кастомізація, платіжні рішення
Професійні асоціації	Організації, що видають збірники для членів	Простота, аналітика, онлайн-доступ	Мобільність, аналітика, простий інтерфейс
Автори	Дослідники, що подають статті	Зручність подачі, прозорість статусу, швидкість	Інтуїтивний UX, сповіщення, трекінг

Кінець таблиці 4.1

Рецензенти	Запрошені експерти	Легкість рецензування, автоматизація	Автоматизовані запити, інтерфейс рецензента
Адміністратори/ІТ	Відповідальні за впровадження і підтримку	Безпека, легке розгортання, інтеграції	Docker, хмара, API, налаштування безпеки

Таблиця 4.2 – Карта позиціонування

Система/Продукт	Технологічна сучасність	Гнучкість/кастомізація	Стислий коментар
Scientific Publishing System	Висока	Висока	Сучасна, модульна, легко налаштовується
Open Journal Systems (OJS)	Середня	Низька/середня	Застарілий інтерфейс, складна кастомізація
ScholarOne, Editorial Manager	Висока	Низька	Потужні, але закриті, дорогі
Manuscript Manager, Scholastica	Середня/висока	Середня	SaaS, але мало локалізації, мало кастомізації

4.3 Оцінка витрат на розробку і підтримку

Розробка та впровадження Scientific Publishing System є комплексним процесом, що потребує ретельного планування ресурсів і фінансів. Оцінка витрат охоплює як початкові інвестиції у створення платформи, так і постійні витрати на її експлуатацію, розвиток і підтримку. Аналіз витрат здійснюється з урахуванням сучасних підходів до розробки програмного забезпечення, використання платних сервісів AWS та організації технічної підтримки.

					КР.ІІЗ-4220.ІІЗ	Арк.
						81
Зм.	Арк.	№ докум.	Підпис	Дата		-

Основна частина бюджету припадає на етап створення MVP (Minimum Viable Product) та подальший розвиток функціоналу системи. Витрати на розробку включають:

- Заробітна плата розробників. Для створення платформи необхідна команда з бекенд- та фронтенд-розробників, DevOps-інженера, QA-фахівця (тестувальника), UI/UX-дизайнера та проектного менеджера.
- Оплата роботи дизайнера. Розробка сучасного, зручного і адаптивного інтерфейсу потребує залучення досвідченого UI/UX спеціаліста.
- Тестування та контроль якості. Передбачає як автоматизоване, так і ручне тестування, написання тест-кейсів, перевірку безпеки й продуктивності.
- Витрати на інфраструктуру для розробки. Це може бути оренда серверів, оплата акаунтів у GitHub, використання систем CI/CD, а також хмарних сервісів для тестових розгортань (наприклад, AWS, DigitalOcean чи Azure).
- Ліцензування інструментів розробки. При необхідності можуть бути витрати на платні IDE, бібліотеки чи компоненти.
- Документація та навчальні матеріали. Створення повноцінної документації для користувачів і адміністраторів, а також підготовка навчальних відео чи презентацій.

Додатково можуть виникати витрати, пов'язані з налаштуванням інфраструктури замовника, міграцією даних зі старих систем, індивідуальною адаптацією під потреби конкретного клієнта, інтеграцією з локальними або зовнішніми сервісами (наприклад, системами оплати, антиплагіат-сервісами, інституційними репозитаріями).

Після запуску системи основними статтями витрат стають:

- Хмарна інфраструктура. Регулярна оплата за використання серверів (наприклад, AWS ECS, RDS, S3), доменних імен, SSL-сертифікатів.

					КР.ІІЗ-4220.ІІЗ	Арк.
						82
Зм.	Арк.	№ докум.	Підпис	Дата		-

Витрати залежать від кількості користувачів та обсягу збережених даних.

- Підтримка та обслуговування. Зарплата технічної підтримки, адміністраторів, оновлення системи безпеки, резервне копіювання, моніторинг працездатності.
- Доопрацювання функціоналу. Постійний розвиток системи відповідно до вимог клієнтів і ринку: додавання нових модулів, інтеграцій, оновлень стандартів, адаптація до змін у законодавстві чи міжнародних вимогах.
- Маркетинг і просування. Частина бюджету виділяється на рекламні кампанії, участь у галузевих конференціях, презентаціях, створення промо-матеріалів.
- Підтримка клієнтів. Консультації користувачів, навчання персоналу замовника, оперативне реагування на звернення й інциденти.
- Ліцензії та сервіси. Витрати на сторонні сервіси (наприклад, антиплагіат, email-розсилки через сервіс типу AWS SES або SendGrid, оплата підписок на бібліотеки чи API).

До неочевидних витрат можна віднести час та ресурси, які витрачаються на підтримку відповідності стандартам безпеки (GDPR, захист персональних даних), аудит коду, сертифікації, а також витрати на юридичний супровід та укладання договорів із клієнтами.

Нижче наведений приклад підрахунку витрат на різноманітні сервіси та команду розробників, які б фактично могли підтримувати проєкт Scientific Publishing System.

					КР.ІІЗ-4220.ІІЗ	Арк.
						83
Зм.	Арк.	№ докум.	Підпис	Дата		-

Таблиця 4.3 – Підрахунок приблизних витрат на сервіси та команду розробників

Стаття витрат	Опис, що входить	Орієнтовна ціна, USD	Коментар
Зарплата розробників	2 розробники (fullstack), 1 DevOps, 1 QA, 1 UI/UX	8 000 – 12 000 / міс	На етапі активної розробки
Домен + SSL-сертифікат	Доменне ім'я, захищене з'єднання	10 – 30 / міс	Залежить від постачальника
Хмарний сервер (AWS/GCP/Azure)	Сервери для backend, файлового сховища, бази даних	150 – 400 / міс	Мінімальна конфігурація
Зберігання файлів (S3/MinIO)	100–200 ГБ для файлів статей, зображень	10 – 50 / міс	AWS S3, Backblaze, DigitalOcean
Інтеграція з DOI (Crossref)	Реєстрація DOI [5]	275 / рік + 1–2 за DOI	Crossref для невеликих журналів
CI/CD, хостинг репозиторіїв (GitHub, GitLab)	Автоматизація розгортання, зберігання коду	0 – 20 / міс	Безкоштовно для open-source
Технічна підтримка	Сапорт, консультації, адміністрування	500 – 1 500 / міс	Залежить від SLA і обсягів
Маркетинг і просування	Реклама, конференції, просування в соцмережах	200 – 600 / міс	Під час активного просування
Ліцензії та підписки на бібліотеки	Платні компоненти, API, бібліотеки	30 – 100 / міс	За потреби

У підсумку сумарні мінімальні місячні витрати для базового SaaS-рішення з невеликою кількістю клієнтів стартують від 1 500–2 500 USD/міс (враховуючи актуальні ціни сервісів) [23]. Реальні цифри залежать від кількості користувачів, масштабів, SLA, інтенсивності маркетингової кампанії й потреб кастомізації.

ВИСНОВКИ

У цій роботі було здійснено комплексний аналіз та розробку сучасної платформи для автоматизації процесів наукового публікування – Scientific Publishing System. Протягом виконання дослідження, на основі ретельного вивчення ринку та тенденцій галузі, було визначено основні вимоги до такого програмного забезпечення, а також враховано потреби усіх основних учасників видавничого процесу: редакцій, авторів, рецензентів та адміністраторів. У ході аналізу були виявлені ключові проблеми, притаманні існуючим рішенням у цій сфері, серед яких особливо варто відзначити складність налаштування, недостатню гнучкість, обмежені можливості локалізації, а також недосконалу інтеграцію з сучасними сервісами.

У процесі проектування системи була розроблена ефективна архітектура платформи, яка забезпечує чітке визначення основних компонентів і сценаріїв взаємодії користувачів. Значну увагу було приділено питанням масштабованості платформи, її безпеці, зручності інтеграції з іншими системами, а також підтримці як локальних, так і міжнародних стандартів. Реалізовано функціональні можливості, які повністю відповідають сучасним вимогам до систем наукового публікування, впроваджено сучасний користувацький інтерфейс, що сприяє автоматизації ключових процесів і забезпечує можливість подальшого розширення функціоналу платформи.

Окремо було проведено оцінку приблизних витрат на створення, впровадження і підтримку платформи. Визначено основні статті бюджету, а також наведено орієнтовні фінансові розрахунки, що дозволяють оцінити економічну доцільність проекту. У рамках роботи було розглянуто цільову аудиторію системи, здійснено сегментацію потенційних користувачів та визначено ключові переваги платформи на сучасному ринку програмних рішень для наукового публікування.

					КР.ІІЗ-4220.ІІЗ	Арк.
						85
Зм.	Арк.	№ докум.	Підпис	Дата		-

Узагальнюючи проведену роботу, можна зробити висновок, що Scientific Publishing System є актуальним, гнучким, масштабованим та водночас простим і зручним у використанні рішенням для автоматизації процесу наукового публікування. Система здатна задовольнити потреби широкого кола користувачів, враховує специфіку галузі та відповідає сучасним вимогам до ефективної організації наукової видавничої діяльності.

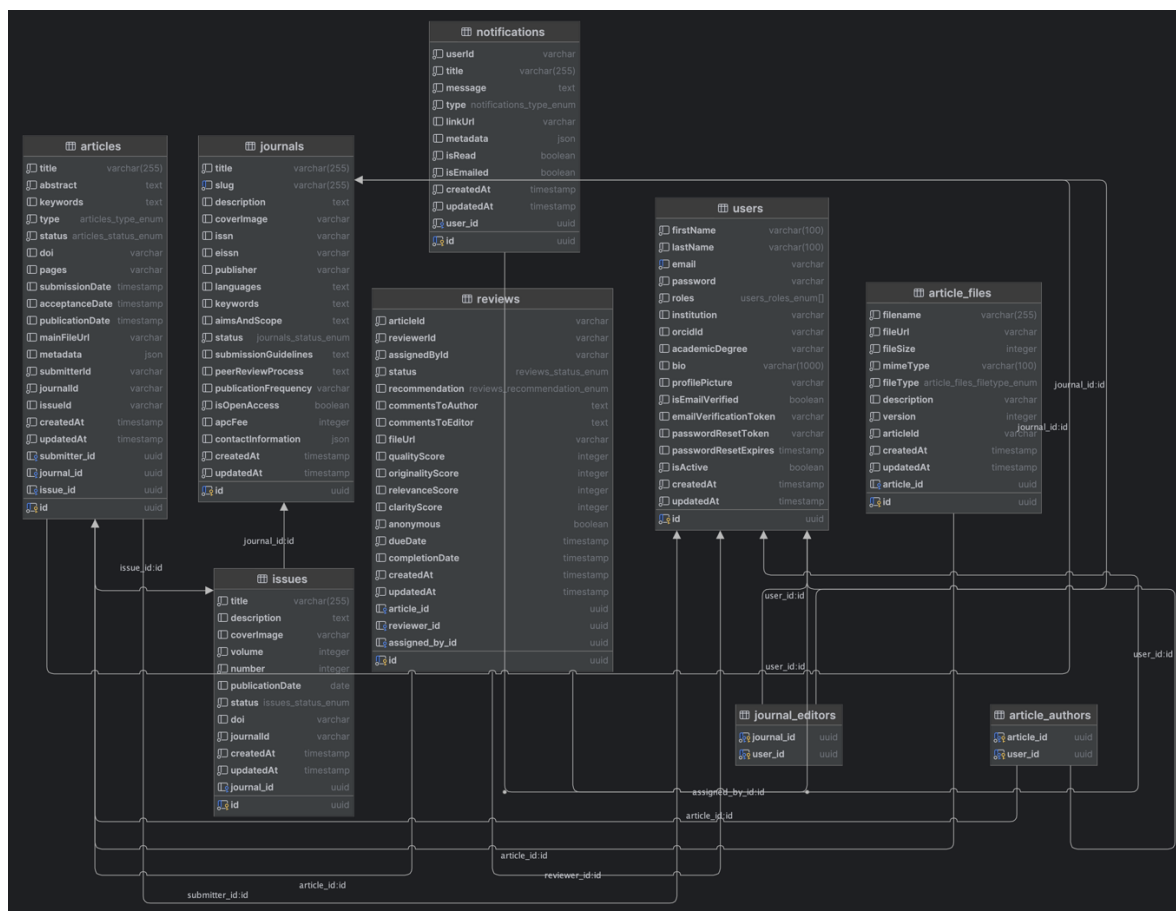
					КР.ІІЗ-4220.ІІЗ	Арк.
						86
Зм.	Арк.	№ докум.	Підпис	Дата		-

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Nielsen, J. Usability Engineering. – Morgan Kaufmann, 1994.
2. Open Journal Systems Documentation [Електронний ресурс]. – Public Knowledge Project. – Режим доступу: <https://docs.pkp.sfu.ca>
3. Public Knowledge Project. OJS: Open Journal Systems [Електронний ресурс]. – Режим доступу: <https://pkp.sfu.ca/ojs/>
4. ORCID. Open Researcher and Contributor ID [Електронний ресурс]. – Режим доступу: <https://orcid.org/>
5. Crossref. Metadata Services [Електронний ресурс]. – Режим доступу: <https://www.crossref.org/education/metadata/>
6. ISO/IEC 27001:2022. Information Security Management Systems – Requirements.
7. NestJS Documentation [Електронний ресурс]. – Режим доступу: <https://docs.nestjs.com/>
8. TypeORM Documentation [Електронний ресурс]. – Режим доступу: <https://typeorm.io/>
9. PostgreSQL. Documentation [Електронний ресурс]. – Режим доступу: <https://www.postgresql.org/docs/>
10. React. Official Documentation [Електронний ресурс]. – Режим доступу: <https://react.dev/>
11. Docker. Documentation [Електронний ресурс]. – Режим доступу: <https://docs.docker.com/>
12. Prometheus and Grafana Monitoring [Електронний ресурс]. – Режим доступу: <https://prometheus.io/docs/>
13. ISO 690:2021. Information and documentation – Guidelines for bibliographic references and citations to information resources.

14. Чорней І. В. Інформаційні системи і технології у видавничій справі. – Львів: Видавництво ЛНУ, 2020.
15. Вишневецький В. П. Проектування інформаційних систем: навч. посіб. – Київ: КНЕУ, 2018.
16. Жежерін Ю. В. Архітектура клієнт-серверних систем. – Харків: ХНУРЕ, 2021.
16. Столяров А. В. Безпека інформаційних систем. – Київ: Видавництво Ліра-К, 2020.
17. Fielding, R. T. Architectural Styles and the Design of Network-based Software Architectures. – University of California, 2000.
18. IEEE Standard Glossary of Software Engineering Terminology. – IEEE Std 610.12-1990.
19. ISO/IEC 25010:2011. Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – System and software quality models.
20. Ghezzi, C., Jazayeri, M., & Mandrioli, D. Fundamentals of Software Engineering. – Prentice Hall, 2002.
21. Sommerville, I. Software Engineering. – 10th ed. – Pearson Education, 2015.
22. Osterwalder, A., Pigneur, Y. Business Model Generation. – Wiley, 2010.
23. Mankins, M. C. How to Make a Business Case. – Harvard Business Review, 2020.

ДОДАТОК А



Додаток А.1 – ERD-діаграма бази даних

	id	firstName	lastName	email	password
1	be5f5984-2c99-4a07-b4d2-b25f44b81443	John	Doe	john.doe1@example.com	\$2b\$10\$TRoxqdu9/AagdZPvD4bAn.nKe0XhEEJbBeSX/bz
2	17fff3fc-22cc-4668-8577-640b477d7f16	John	Doe	john@example.com	\$2b\$10\$IjLWhjtRKUop9T.JYCjke.HFoxHSBhSkQRdLK3
3	4e582ce2-e4b2-49fe-b9be-c994022ecd30	John	Doe	john3@example.com	\$2b\$10\$ByLZtcKnCx6p6wLFA.Na/uJiIhvnMKGxeW7N6Ej
4	bbc1fffa-dfb5-4887-9221-f08416810413	John	Doe	asd@example.com	\$2b\$10\$p8tVA8XI6yozIYcaUETwC.oaTEjZL8sEpDxvHjp
5	647b385b-66bb-459d-8777-b569a8ee5e83	Admin	User	admin@example.com	\$2b\$10\$pzF3UrDnc1gGEb341qjgKreQ40FUNAWybmDQuFuU
6	bec536f6-4541-469b-b139-323011de4370	John	Updated	john.doe@example.com	\$2b\$10\$/rvcxEV071nnc1zjeYQpuEE.kfh16c6ZP7QCvt
7	6295d500-ec95-4039-bbf4-1da8698f23ff	John	Doe	doe1@example.com	\$2b\$10\$H4.J.aA0VTA1LP6Hof79F0yDLUBMegdWMgqzW0T
8	f6b36475-d37b-4d2f-af36-750d3659d58a	John	Doe	doe3@example.com	\$2b\$10\$jqzQZnZvAisI5itU2vXzA0QqLLCw2vPRbWAbx0k
9	dd84aae7-e82a-4e58-98a6-b427c0bd39e7	Admin	User	admin1@example.com	\$2b\$10\$B6ZTVJqd5CvLjXx/Au3FH.UmxZjXpNF9CzsWWgY
10	4778f1a3-b558-41f3-b4ec-f18dd576a191	John	Updated	john1@example.com	\$2b\$10\$K6GV.tcDR9u/ohMXFLp0TeEofvP8jEhBVP0dqyA
11	f9229b64-36a3-4543-bdda-c401e7528412	John	Doe	doe2@example.com	\$2b\$10\$Uk0adNgdyTaVwLX7Vw42N0f0ex6dxTLDEcThBS0

Додаток А.2 – Заповнення таблиці users даними користувачів

ДОДАТОК Б

auth		
POST	/api/auth/login	Login user
POST	/api/auth/register	Register a new user
GET	/api/auth/verify-email	Verify user email
POST	/api/auth/forgot-password	Request password reset
POST	/api/auth/reset-password	Reset password using token
GET	/api/auth/profile	Get current user profile

Рисунок Б.1 – ендпоінти auth

users		
POST	/api/users	Create a new user
GET	/api/users	Get all users
GET	/api/users/{id}	Get a user by ID
PUT	/api/users/{id}	Update a user
DELETE	/api/users/{id}	Delete a user

Рисунок Б.2 – ендпоінти users

journals		
POST	/api/journals	Create a new journal
GET	/api/journals	Get all journals
GET	/api/journals/{id}	Get a journal by ID
PATCH	/api/journals/{id}	Update a journal
DELETE	/api/journals/{id}	Delete a journal
GET	/api/journals/by-slug/{slug}	Get a journal by slug
POST	/api/journals/issues	Create a new issue
GET	/api/journals/{journalId}/issues	Get all issues for a journal
GET	/api/journals/issues/{id}	Get an issue by ID
PATCH	/api/journals/issues/{id}	Update an issue
DELETE	/api/journals/issues/{id}	Delete an issue

Рисунок Б.3 – ендпоінти journals

reviews		
POST	/api/reviews	Create a new review assignment
GET	/api/reviews	Get all reviews with filtering options
GET	/api/reviews/{id}	Get a review by ID
PATCH	/api/reviews/{id}	Update a review
DELETE	/api/reviews/{id}	Delete a review
GET	/api/reviews/summary/{articleId}	Get review summary for an article

Рисунок Б.4 – ендпоінти reviews

ДОДАТОК В

```

import {
  Injectable,
  UnauthorizedException,
  BadRequestException,
  ConflictException,
  Logger,
} from '@nestjs/common';
import { JwtService } from '@nestjs/jwt';
import { ConfigService } from '@nestjs/config';
import { UsersService } from '../users/users.service';
import { LoginDto } from '../dto/login.dto';
import { RegisterDto } from '../dto/register.dto';
import { User } from '../users/entities/user.entity';

@Injectable()
export class AuthService {
  private readonly logger = new Logger(AuthService.name);

  constructor(
    private usersService: UsersService,
    private jwtService: JwtService,
    private configService: ConfigService,
  ) {}

  async validateUser(email: string, password: string):
  Promise<User> {
    const user = await
  this.usersService.findByEmailAndPassword(email);
    if (!user) {
      throw new UnauthorizedException('Invalid credentials');
    }

    const isPasswordValid = await
  user.validatePassword(password);
    if (!isPasswordValid) {
      throw new UnauthorizedException('Invalid credentials');
    }

    // Опціонально перевіряємо чи верифікована пошта, залежно від
  налаштувань
    const requireVerification =
  this.configService.get('REQUIRE_EMAIL_VERIFICATION') === 'true';
    if (requireVerification && !user.isEmailVerified) {
      throw new UnauthorizedException('Email not verified');
    }

    if (!user.isActive) {
      throw new UnauthorizedException('User account is
  deactivated');
    }

    return user;
  }

```

```

    }

    async login(loginDto: LoginDto) {
      const user = await this.validateUser(loginDto.email,
loginDto.password);
      const payload = {
        sub: user.id,
        email: user.email,
        roles: user.roles,
      };

      return {
        user: {
          id: user.id,
          email: user.email,
          firstName: user.firstName,
          lastName: user.lastName,
          roles: user.roles,
        },
        accessToken: this.jwtService.sign(payload),
        expiresIn: this.configService.get('JWT_EXPIRATION'),
      };
    }

    async register(registerDto: RegisterDto) {
      this.logger.log(registerDto);
      try {
        // Перевіряємо, чи існує користувач з такою електронною
поштою
        const existingUser = await
this.usersService.findByEmail(registerDto.email);
        if (existingUser) {
          throw new ConflictException('User with this email already
exists');
        }

        // Створюємо нового користувача
        const newUser = await this.usersService.create({
          ...registerDto,
          isEmailVerified: false, // за замовчуванням email не
підтверджений
        });
        this.logger.log(newUser);

        // Зараз тут ми б могли відправити лист підтвердження

        // Повертаємо відповідь
        return {
          message: 'User registered successfully. Please verify
your email.',
          userId: newUser.id,
        };
      } catch (error) {
        this.logger.error(`Registration error: ${error.message}`,
error.stack);
      }
    }
  }
}

```

```

        if (error instanceof ConflictException || error instanceof
BadRequestException) {
            throw error;
        }
        throw new BadRequestException(`Registration failed:
${error.message}`);
    }
}

async verifyEmail(token: string) {
    try {
        const user = await this.usersService.verifyEmail(token);

        return {
            message: 'Email verified successfully',
            userId: user.id,
        };
    } catch (error) {
        this.logger.error(`Email verification error:
${error.message}`, error.stack);
        throw new BadRequestException('Email verification failed');
    }
}

async forgotPassword(email: string) {
    try {
        const token = await
this.usersService.createPasswordResetToken(email);
        // Here we would send a password reset email
        return {
            message: 'Password reset instructions sent to your
email',
        };
    } catch (error) {
        this.logger.error(`Password reset request error:
${error.message}`, error.stack);
        // Не розкриваємо, чи існує користувач з такою електронною
ПОШТОЮ
        return {
            message: 'If an account with this email exists, password
reset instructions have been sent',
        };
    }
}

async resetPassword(token: string, password: string) {
    try {
        if (!token || !password) {
            throw new BadRequestException('Token and password are
required');
        }

        await this.usersService.resetPassword(token, password);

        return {

```

```
        message: 'Password reset successful',
    };
    } catch (error) {
        this.logger.error(`Password reset error: ${error.message}`,
error.stack);
        throw new BadRequestException('Password reset failed');
    }
}
```

Код файлу auth.service.ts